

STUDY GUIDE · 30 PAGES

Reinforcement Learning

From Q-learning to deep Q-networks and policy gradients

A companion to the freeCodeCamp video
“Reinforcement Learning Course - Full Machine Learning Tutorial”
taught by Phil Tabor (Machine Learning with Phil / neuralnet.ai) · 3 h 55 min

5

parts

25

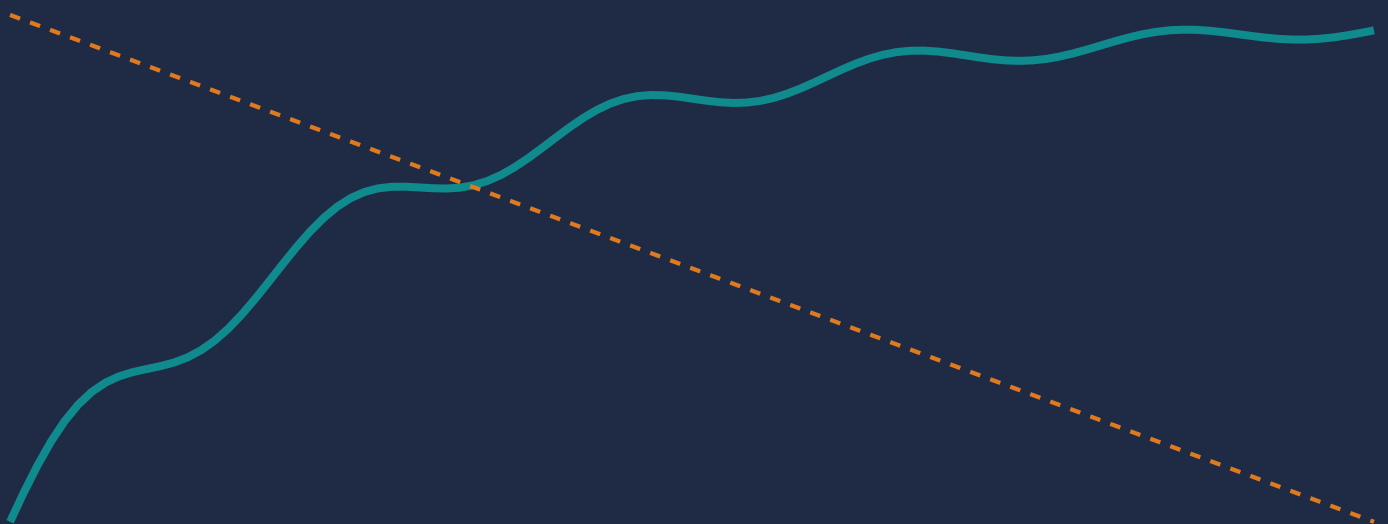
sections

9

algorithms & demos

5

environments



Built from the video's transcript, on-screen slides/equations/plots (OCR + frame inspection) and clearly-marked extra knowledge.
Independent study aid - all course content belongs to its author. Compiled October 2026.

Contents & how to use this guide

Video: whole course, 3 h 55 min

This guide condenses the **freeCodeCamp “Reinforcement Learning Course - Full Machine Learning Tutorial”** (instructor: Phil Tabor) into 30 pages you can study, search and revise from. The video is really a playlist of tutorials taught *top-down* (deep methods first, basics last). This guide re-orders them *bottom-up* – foundations → tabular methods → deep Q-learning → policy gradients – and tells you the video time-stamp for every section so you can jump between the two.

#			
PART I · FOUNDATIONS			
1	What is reinforcement learning?	3:08 – 3:11	3
2	Rewards, policies & vocabulary	3:11 – 3:14	4
3	Markov decision processes, returns & discounting	3:17 – 3:21	5
4	Value functions, Bellman equation, model-based vs model-free	3:14 – 3:23	6
5	Explore vs. exploit	3:23 – 3:29	7
✓			
PART II · TABULAR METHODS			
6	Q-learning	0:01 – 0:08 · 2:57 – 3:05	9
7	Build your own environment: Grid World design	2:34 – 2:42	10
8	Grid World implementation	2:42 – 2:55	11
9	Q-learning agent on Grid World	2:55 – 3:08	12
10	SARSA	3:29 – 3:32	13
11	SARSA on CartPole	3:32 – 3:40	14
12	Maximization bias & Double Q-learning	3:40 – 3:43	15
13	Double Q-learning in code	3:43 – 3:54	16
✓			
PART III · DEEP Q-LEARNING			
14	From tables to neural networks: DQN overview	0:01 – 0:08 · 0:52 – 0:53	18
15	DQN ingredients: memory, ϵ -greedy, targets, preprocessing	0:08 – 0:49 · 1:06 – 1:28	19
16	DQN in TensorFlow (Breakout): the network	0:09 – 0:21	20
17	DQN in TensorFlow (Breakout): the agent	0:21 – 0:49	21
18	DQN in PyTorch (Space Invaders): network & agent	0:52 – 1:15	22
19			
PART IV · POLICY GRADIENTS			
20	Policy gradients: the idea and the math	1:46 – 1:50	24
21	Why policy gradients? Variance & sample efficiency	1:50 – 1:55	25
22	Lunar Lander with policy gradients (TensorFlow)	1:55 – 2:21	26
23			
PART V · WRAP-UP			
24	Comparison, decision guide & hyperparameter cheat-sheet	3:54 + synthesis	28
25	Practice: exercises, quiz & where next	3:54 – 3:55	29
R	Reference: glossary, formulas, modern-library notes		30

Where the material comes from

- **Transcript** – the auto-generated captions of the whole video ($\approx 30,000$ words). Numbers (learning rates, layer sizes) were cross-checked against on-screen text where possible; auto-captions occasionally mis-hear digits.
- **On-screen frames** – slides, equations and result plots were pulled out of the video and are reproduced as figures. The file is only 144p, so *code on screen is not legible*: code listings are rebuilt from the narration, and the tabular ones (Grid World, Q-learning) were actually run to confirm the results the instructor reports.
- **Extra knowledge** – clearly marked **BEYOND THE VIDEO**. Other call-outs: **KEY IDEA**, **WATCH OUT**, **FROM THE VIDEO**.

1

What is reinforcement learning?

Video: 3:08 – 3:11 (+ 0:02 intro) · Part I · Foundations

Most machine-learning you meet first is **supervised learning**: show the network thousands of examples, each hand-labelled with the right answer. It works (computer vision, regression) but needs mountains of labelled data, which is usually only available to large companies. People do not learn like that. We learn by **doing** – trying actions, seeing what happens and adjusting. Strip that idea down and three ingredients remain: an **environment** that reacts, **actions** that affect it, and the thing that learns – the **agent**. No labelled dataset is required.

one step of interaction, repeated until the episode ends

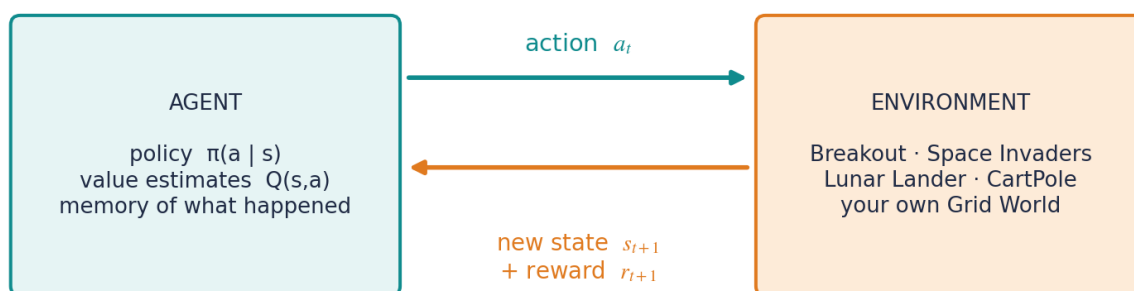


Figure 1 – The agent-environment loop (redrawn from the idea the instructor explains). Every algorithm in the course is a different way of filling in the left-hand box.

Reinforcement learning in one paragraph

The environment can be anything from a text card game to an Atari game to (in principle) the real world. The agent acts through a set of actions – usually discrete, e.g. *move left*, *move right*, *fire*. Each action causes the environment to move from one **state** to another and hands the agent a **reward** (or penalty). The agent keeps track of the sequence of states, actions and rewards, looks for patterns, and tries to **maximise total reward over time** – not just the next reward. That last phrase is what makes the problem interesting.

KEY IDEA Reinforcement learning = learning to choose actions that maximise **long-term** reward by interacting with an environment. No labels, only a reward signal.

The motivating example from the video: three job offers

The course opens with a puzzle. You have offers from three companies. Each pays a starting salary, promises a raise later, and carries a probability that you will quit or wash out before the raise arrives (figures as shown in the on-screen caption):

Company	Starting salary	Raise	When	P(quit / wash out)
Facebook	\$250,000	10 %	after 3 years	40 %
Google	\$200,000	20 %	after 3 years	25 %
Amazon	\$350,000	5 %	after 5 years	60 %

The highest starting salary is not obviously the best choice: future raises are uncertain and every decision changes what you can earn later. This is exactly the structure of an RL problem – “*how can an agent maximise long-term rewards in an environment with uncertain outcomes?*” The rest of the course builds the tools to answer that, starting with Q-learning, which maps (state, action) pairs to the future reward the agent expects.

How RL differs from supervised learning

	Supervised learning	Reinforcement learning
Feedback	Correct answer for every example	A scalar reward, often delayed
Data	Fixed, pre-labelled dataset	Generated by the agent’s own interaction
Goal	Minimise prediction error	Maximise cumulative reward
Examples in this course	—	Breakout, Space Invaders, Lunar Lander, CartPole, Grid World

FROM THE VIDEO The instructor stresses that Q-learning updates *after every time step*, whereas policy-gradient methods update at the end of an episode – a distinction that returns in Parts II and IV.

2

Rewards, policies & vocabulary

Video: 3:11 – 3:14 · Part I · Foundations

The cast of characters

Term	Meaning	Example in the course
Environment	Everything the agent interacts with; reacts to actions.	Atari Breakout, Lunar Lander, your own Grid World
State s	The environment's situation as the agent observes it.	A stack of 4 frames; an 8-number vector; square number 0-80
Action a	What the agent can do (usually a discrete set).	left / right / fire; up / down / left / right
Reward r	Number returned after each action – the only “teacher”.	–1 per Grid World step; +1 per CartPole step
Policy $\pi(a s)$	Probability of choosing action a when in state s .	ϵ -greedy table lookup; a softmax output layer
Agent	The software that stores transitions and looks for patterns that maximise reward.	Q-table, DQN, policy network

State transitions can be random

An action does not have to do what you expect. In Space Invaders, pressing “left” moves the ship left with probability 1. In the **Frozen Lake** environment, trying to move left can actually send the agent right, up or down. So the environment has **state-transition probabilities** – the chance of landing in state s' and receiving reward r after taking action a in state s . They need not be 100 %, only sum to 1 over the possible outcomes.

WATCH OUT The **policy** probabilities (how the agent picks actions) are not the same thing as the **state-transition** probabilities (how the environment responds). Beginners mix these up constantly.

Reward design is the most important design decision

If you remember one thing from the introduction, remember this: **every RL algorithm maximises the reward you give it – nothing more, nothing less.** The instructor makes two points with examples:

- **Safety.** The danger of AI is not malice but being *ruthlessly rational*. The classic paper-clip story: an agent rewarded only for paper clips notices that humans consume resources it could use – so humans become an obstacle. Reward design must include penalties for actions that endanger people.
- **Inefficiency.** In chess you might penalise losing pieces, but that would stop the agent discovering *gambits* where a sacrifice pays off later. AlphaZero is notorious for sacrificing several pawns and still beating the strongest traditional engines. Over-specifying the reward can block better strategies.
- **Practical echo in the code.** In the PyTorch Space Invaders agent the author adds an extra penalty when the last life is lost, so the agent learns “losing is really bad” – a deliberate reward-shaping choice.

Policy and the Bellman equation (preview)

The algorithm that decides how the agent acts in any state is its **policy**, written $\pi(a|s)$: “the probability of selecting action a given the environment is in state s ” (this is the on-screen definition). The mathematical relationship linking state transitions, rewards and the policy is the **Bellman equation**; it gives the *value* – the expected future reward – of a state under a policy. Most RL algorithms are ways of maximising or solving it (Section 4).

KEY IDEA Three questions to ask of any RL problem: **What are the states? What are the actions? What exactly is rewarded?** Getting the third wrong gives you an agent that optimises the wrong thing very efficiently.

3

Markov decision processes, returns & discounting

Video: 3:17 – 3:21 · Part I · Foundations (course “Module 2”)

Each decision an agent makes influences not just its immediate reward but *all future rewards*. Written down, you get a sequence of states, actions and rewards – a **decision process**. If each state depends only on the previous state and the agent's action (not on the longer history), it is a **Markov decision process (MDP)**. This is an idealised abstraction: real problems break it to varying degrees, but it is usually “good enough” to apply the mathematics of RL.

Return: what the agent actually tries to maximise

From the agent's side it receives a stream of rewards. The **return** at time t is the sum of rewards from $t+1$ up to the final time step T :

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

Reaching the final step ends an **episode** (a discrete period of play) and drops the agent into a unique **terminal state**. Whatever way the episode ends, the terminal state is the same and **its expected future reward is exactly zero** – nothing follows. Tasks that split into episodes are **episodic tasks** (a Breakout game, a Lunar Lander landing). Tasks without a natural end are **continuing tasks**; with $T = \infty$ the sum could be infinite and “maximise it” becomes meaningless.

Discounting fixes both problems

The fix – used for episodic and continuing tasks alike – is **discounting**: the agent values later rewards less and less, following a power law controlled by the **discount rate γ** ($0 \leq \gamma \leq 1$, a hyper-parameter).

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Factor out γ and a useful fact appears: **returns at successive time steps are recursively related**. This recursion is exploited constantly in RL (it is the seed of the Bellman equation and of every update rule in Part II):

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Worked examples (to build intuition)

1 · Grid World ($\gamma = 1$). Every step costs -1 and the final step costs 0 . The best possible path takes 12 moves, so the return is -11 – exactly the value the agent reaches in Section 9.

2 · Discounted reward. Rewards of $+1$ for five steps with $\gamma = 0.9$:

$$G_0 = \sum_{k=0}^4 0.9^k \cdot 1 = \frac{1 - 0.9^5}{1 - 0.9} \approx 4.095$$

3 · CartPole with $\gamma = 0.9$ (beyond the video). A perfect 200-step episode earns $+1$ each step, so the discounted return is only about 10:

$$\sum_{k=0}^{199} 0.9^k = \frac{1 - 0.9^{200}}{0.1} \approx 10$$

Rewards after roughly step 40 hardly matter, which is a plausible reason γ matters so much in the Double-Q experiments (Section 13). The instructor argues $\gamma = 1$ is more natural there because the dynamics are deterministic.

KEY IDEA Return = discounted sum of future rewards. $\gamma \approx 0$: short-sighted agent. $\gamma = 1$: perfectly far-sighted (counts all future rewards equally – only safe in episodic tasks). Discounting is justified mainly by *uncertainty* about whether future rewards will really arrive.

FROM THE VIDEO Slides shown in this module: the return formula $G_t = R_{t+1} + R_{t+2} + \dots + R_T$, then the discounted version, then the recursion – exactly the three equations above.

4

Value functions, the Bellman equation & model-based vs model-free

Video: 3:14 – 3:23, 3:29 – 3:32 · Part I · Foundations

Because the agent follows a policy, it can use the sequence of states, actions and rewards to work out how good each situation is. Two functions formalise “how good”:

Value function and action-value function

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

$v_{\pi}(s)$ – the *value of a state*: the expected return when starting in s and following policy π thereafter.

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

$q_{\pi}(s, a)$ – the *action value*: the expected return for taking action a in state s and following π afterwards. Q-learning (Part II) works with this function – the “Q” in Q-learning is just the nomenclature for the value of a state-action pair.

The Bellman equation

Plug the recursive form of the return into v_{π} and the value function itself turns out to be recursive. That is the **Bellman equation**:

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

It is an **expectation**: a weighted average over (i) how likely the agent is to pick each action and (ii) how likely each next state and reward are, given the transition probabilities. In words: *the value of a state is the immediate reward plus the discounted value of where you land next, averaged over everything that could happen*. The rest of the course is largely about different schemes to solve it and improve the policy so that value increases over time.

Do you know the environment's rules?

To have a **model** of the environment means knowing all the state-transition probabilities with certainty. That is rare. The algorithm families split accordingly:

Family	Needs a model?	How it learns	Examples
Dynamic programming	Yes – full transition probabilities	Directly solves the Bellman equation using the known model	Policy / value iteration (limited real-world use)
Model-free, tabular	No	Learns from experience; stores values in a table	Q-learning, SARSA, Double Q-learning
Model-free, deep	No	Learns from experience; a neural network approximates the function	Deep Q-learning (DQN), policy gradients

Two properties of the model-free methods in this course

- **Bootstrapping** – estimates are used to generate other estimates, so you can start from arbitrary guesses (the instructor's words: “wild guesses”) and refine. Only terminal states must start at exactly 0.
- **Temporal-difference (TD) learning** – Q-learning and SARSA update after every time step instead of waiting for the episode to finish.
- **Off-policy vs on-policy** – see Section 5: Q-learning learns about the greedy policy while behaving ϵ -greedily; SARSA learns about the policy it is actually following.

KEY IDEA Value = expected **future** return. The agent never sees values directly – it *estimates* them from experience, and the quality of those estimates decides how good its decisions are.

5

Explore vs. exploit

Video: 3:23 – 3:29 · Part I · Foundations (course “Module 3”)

Should the agent take the action it *knows* pays best right now, or try something else that might pay more? That conflict – **short-term reward vs. exploration for long-term reward** – is the explore-exploit dilemma, and every RL algorithm must deal with it. The course shows three tools.

1 · Optimistic initial values

The starting Q-values are arbitrary, but if you know the reward structure you can choose them to *force* exploration. In Grid World every step gives -1 , so true values are always ≤ 0 . Initialise every Q to 0 (“optimistic”): the first time the agent tries an action it receives -1 and is *disappointed* – it expected chocolate cake and got mud. Next time in that state the untried actions still look like 0, which beats -1 , so it tries those. Over time every state-action pair is explored without any explicit randomness.

2 · ϵ -greedy action selection

Most of the time pick the best-known (greedy) action; with probability ϵ pick a random one. ϵ is the fraction of time spent exploring – a hyper-parameter between 0 and 1. Start near 1 (almost fully random) and **decay** it so the agent becomes mostly greedy. The shape of the decay (linear, power law, ...) is “not critically important”.

ϵ -greedy in four lines (the course writes the test as `random < 1 -  $\epsilon$` for the greedy branch – equivalent)

```
def choose_action(Q, state, actions, eps):
    if np.random.random() < eps:           # explore
        return np.random.choice(actions)
    values = np.array([Q[state, a] for a in actions]) # exploit: best known action
    return actions[np.argmax(values)]         # (ties -> first element)
```

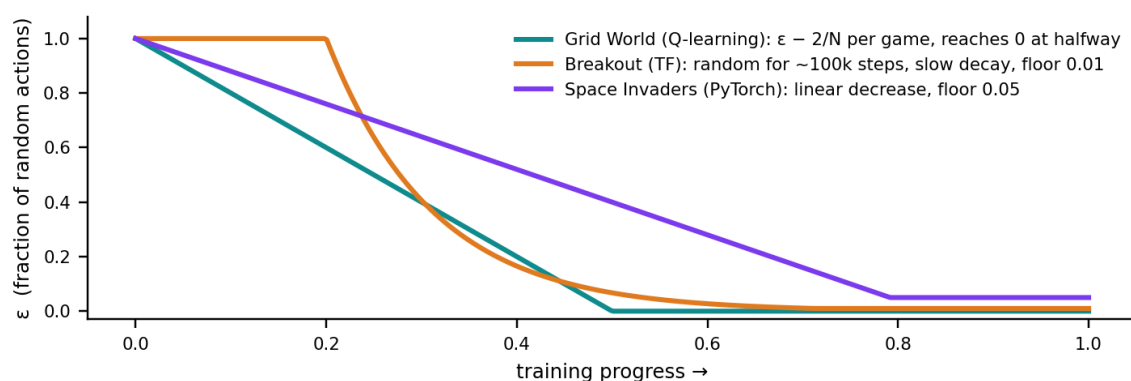


Figure 2 – The ϵ schedules used in the course (shapes are illustrative; each curve follows the description in the matching tutorial).

When should ϵ reach zero?

- **Simple, fully-understood environments** (Grid World, where the optimum is known): let ϵ decay to 0 – converge to a purely greedy strategy.
- **Complex games** (Space Invaders and friends): there are so many variables that you cannot be sure the strategy is optimal, so keep ϵ at a **small finite value** (the course uses 0.01 and 0.05) to keep testing the agent’s model of the world. Never set it to exactly zero.

3 · Two policies: on-policy vs. off-policy

So far one policy both *generated* actions and was *updated*. Nothing requires that. An agent can use one policy to generate behaviour and the resulting data to update the value function of **another** policy: **off-policy learning**. Q-learning does exactly this – it behaves ϵ -greedily but updates the estimate for the *purely greedy* policy. (Using the same policy for both is **on-policy**; SARSA does that – Section 10.)

$$P(\text{greedy action}) = 1 - \epsilon + \frac{\epsilon}{|A|}$$

Beyond the video: with an ϵ -greedy rule that picks uniformly among all $|A|$ actions when exploring, the greedy action is actually chosen with probability $1 - \epsilon + \epsilon/|A|$ – the instructor makes the same point for 6 actions in Space Invaders.



Checkpoint I - Foundations

Video: recap of 3:08 – 3:29 · Part I

Concept	One-line definition	Symbol
State / action / reward	What the agent sees / can do / is told after each action	s, a, r
Policy	How the agent picks actions: probability of a given s	$\pi(a s)$
Episode	One run from start to terminal state; terminal value is 0	—
Return	Discounted sum of future rewards	G_t
Discount rate	How much future rewards are worth today (0 ... 1)	γ
State value	Expected return from state s following π	$v_{\pi}(s)$
Action value	Expected return after taking a in s , then following π	$q_{\pi}(s,a) / Q(s,a)$
Bellman equation	Value = reward + discounted value of next state, averaged over the dynamics	—
MDP	Next state depends only on the current state and action (Markov property)	—
Model-free	Learns from experience without knowing transition probabilities	—
Bootstrapping	Updating estimates from other estimates	—
ϵ -greedy	Explore with probability ϵ , otherwise take the greedy action	ϵ
Optimistic init.	Start Q too high so disappointment drives exploration	—
On / off-policy	Learn about the policy you follow / about a different (greedy) one	—

Common confusions

- **Reward $\neq$ return $\neq$ value.** Reward is one number from one step; return is the (discounted) sum of rewards along one trajectory; value is the *average* return you expect.
- **Policy probabilities $\neq$ transition probabilities.** The first is the agent's choice, the second is the environment's behaviour.
- **$\gamma = 1$ is not "wrong".** With a deterministic environment and finite episodes (Grid World, CartPole) there is nothing to discount for – the instructor even argues 0.9 in CartPole is unnecessary.
- **ϵ is not "the learning rate".** ϵ controls *behaviour* (exploration). α controls *how fast estimates move*. Different jobs.
- **"Maximise reward" is literal.** The agent optimises exactly what you reward, not what you meant.

Try it by hand (5 minutes)

A Grid World episode gives rewards $-1, -1, -1, -1, 0$ (the last step enters the exit). **(a)** What is G_0 with $\gamma = 1$? **(b)** With $\gamma = 0.5$? **(c)** Use the recursion $G_t = R_{t+1} + \gamma G_{t+1}$ to confirm (b) working *backwards* from the end.

Answers: (a) -4 . (b) $-1 - 0.5 - 0.25 - 0.125 + 0 = -1.875$. (c) $G_4 = 0 \rightarrow G_3 = -1 \rightarrow G_2 = -1 + 0.5(-1) = -1.5 \rightarrow G_1 = -1 + 0.5(-1.5) = -1.75 \rightarrow G_0 = -1 + 0.5(-1.75) = -1.875 \checkmark$.

KEY IDEA You now have the vocabulary for everything that follows. Part II turns these definitions into code: a single table of Q -values, updated after every step.

6

Q-learning

Video: 0:01 – 0:08 and 2:57 – 3:05 · Part II · Tabular methods

Q-learning works by **mapping (state, action) pairs to the future reward the agent expects** to receive. It picks actions with ϵ -greedy selection, takes the action, receives its reward from the environment – and then **learns immediately, after every time step** (a form of temporal-difference learning), rather than at the end of an episode as policy-gradient methods do.

The table

In its traditional form Q-learning literally keeps a **table** of state-action pairs. In Python this can be a dictionary with (state, action) tuples as keys. The agent needs no history of what happened – it updates the entry in place as it plays. This is feasible only when the number of states and actions is small and discrete.

	up	down	left	right
state 0	−14.2	−11.0	−14.9	−12.1
state 1	−13.8	−10.9	−12.3	−13.0
...	...	...	...	...
state 79	−2.0	−2.0	−1.0	−1.0

Illustrative Q-table (made-up numbers): one row per state, one column per action; each cell is the agent's current estimate of the discounted future reward.

The update rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Read it left to right: **new estimate = old estimate + α × (what we just observed – what we believed)**. The bracket is the **TD error**: the difference between the reward-plus-best-future-value we actually saw ($r + \gamma \max Q(s', a')$) and the value we had stored for the action we took. The update biases the table, over time, towards actions that produce the best outcomes.

- α (learning rate) – how far each update moves the estimate; the course uses 0.1 for tabular problems.
- γ – discount factor; $\max_{a'} Q(s', a')$ – the value of the *best* action in the next state, whether or not the agent will really take it (this is why Q-learning is **off-policy**).
- **Terminal states** have value 0, so on the last step the target is just r .

One update, by hand

Grid World, $\alpha = 0.1$, $\gamma = 1$. The agent is in a state where every Q is still 0, takes a step and gets $r = -1$; the next state's best Q is also 0:

$$Q \leftarrow 0 + 0.1 [(-1) + 1.0 \cdot 0 - 0] = -0.1$$

Next time the agent is in this state this action looks worse (-0.1) than its untried siblings (0), so it tries another – the optimistic-initialisation effect from Section 5 in action.

Algorithm

```

initialise  $Q(s, a) = 0$  for all states and actions (terminal states must stay 0)
for each episode:
     $s = \text{env.reset}()$ 
    while not done:
         $a = \text{epsilon\_greedy}(Q, s)$                 # behaviour policy
         $s', r, \text{done} = \text{env.step}(a)$ 
         $a^* = \text{argmax}_{a'} Q(s', a')$                 # greedy target action (not necessarily taken)
         $Q(s, a) += \alpha * (r + \gamma * Q(s', a^*) - Q(s, a))$ 
         $s = s'$ 
    decay epsilon

```

WATCH OUT A table cannot cope with huge or continuous state spaces (an Atari screen has far too many possible images). The fix in Part III is to replace the table by a **neural network** – deep Q-learning.

7

Build your own environment: Grid World design

Video: 2:34 – 2:42 · Part II · Tabular methods

A subscriber asked: *how do I create my own RL environment?* The answer is two tutorials: first an environment, then an agent. The environment copies the **OpenAI Gym interface**, so any gym-style algorithm can run on it. It needs only NumPy and matplotlib – no GPU.

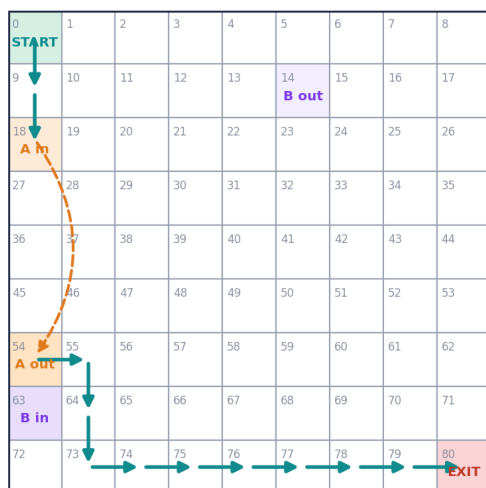


Figure 3 – The 9×9 Grid World. Orange: teleporter A (18 → 54, a shortcut). Purple: teleporter B (63 → 14, a setback). Teal arrows: the optimal route found in Section 9.

The specification

- An $m \times n$ grid (the course uses $9 \times 9 = 81$ squares, numbered 0 ... 80 row by row).
- The agent starts in the **top-left** (square 0) and must reach the **bottom-right** (square 80) – the terminal state.
- Reward **–1 for every step**, **0** on the step that reaches the terminal state. So the agent maximises reward by *minimising the number of steps*.
- Two **magic squares** teleport the agent: **A** at 18 sends it forward to 54; **B** at 63 sends it back to 14. The point is to test whether the agent discovers the shortcut and learns to avoid the trap.
- Moves off the edge are illegal: the agent stays put and still gets –1 (“wasting time”).

Vocabulary borrowed from Sutton & Barto

The author follows the notation of Sutton and Barto’s *Reinforcement Learning: An Introduction* (recommended in the video):

Name	Meaning	Grid World value
state space S	all states <i>except</i> the terminal state	[0 ... 79]
state space plus S⁺	all states <i>including</i> the terminal state	[0 ... 80]
terminal test	a state is terminal iff it is in S⁺ but not in S – set difference gives it for free	state 80
action space	dictionary mapping each action to its change in square number	{U: –9, D: +9, L: –1, R: +1}
possible actions	a plain <i>list</i> [U, D, L, R] – handy because Q-learning picks actions at random	4 actions

What every gym-compatible environment needs

Method	Job	Returns
reset()	put the agent back at the start, rebuild the grid	initial state (0)
step(action)	apply the move, handle teleporters, check legality, compute reward	(new state, reward, done, info)
render()	print the grid as text for debugging (“X” marks the agent)	nothing

FROM THE VIDEO The instructor’s checklist for any new environment: an initialiser, a state space and state space plus, a way to denote possible actions, a way to make sure a move is legal, a step function that returns (position, reward, terminal flag, debug info), plus reset and print/render functions.

KEY IDEA Why teleporters? They create a **shortcut that a good agent must discover**. Without it the best path is just 16 steps; with A it drops to 12. The shortcut makes it obvious on a plot whether learning worked.

8

Grid World implementation

Video: 2:42 – 2:55 · Part II · Tabular methods

gridworld.py - reconstructed from the tutorial and tested (see Section 9 for the result)

```

class GridWorld:
    def __init__(self, m, n, magic_squares):
        self.grid = np.zeros((m, n))
        self.m, self.n = m, n
        self.state_space = [i for i in range(m * n)]
        self.state_space.remove(m * n - 1)  # terminal state removed
        self.state_space_plus = [i for i in range(m * n)]
        self.action_space = {'U': -m, 'D': m, 'L': -1, 'R': 1}
        self.possible_actions = ['U', 'D', 'L', 'R']
        self.magic_squares = magic_squares
        self.agent_position = 0
        self.add_magic_squares()

    def add_magic_squares(self):
        i = 2
        for square in self.magic_squares:
            x, y = square // self.m, square % self.n
            self.grid[x][y] = i
            i += 1
            x, y = self.magic_squares[square] // self.m, self.magic_squares[square] % self.n
            self.grid[x][y] = i
            i += 1

    def is_terminal_state(self, state):
        return state in self.state_space_plus and state not in self.state_space

    def get_agent_row_and_column(self):
        return self.agent_position // self.m, self.agent_position % self.n

    def set_state(self, state):
        x, y = self.get_agent_row_and_column()
        self.grid[x][y] = 0
        self.agent_position = state
        x, y = self.get_agent_row_and_column()
        self.grid[x][y] = 1

    def off_grid_move(self, new_state, old_state):
        if new_state not in self.state_space_plus:
            return True
        elif old_state % self.m == 0 and new_state % self.m == self.m - 1:
            return True
        elif old_state % self.m == self.m - 1 and new_state % self.m == 0:
            return True
        return False

    def step(self, action):
        resulting_state = self.agent_position + self.action_space[action]
        if resulting_state in self.magic_squares:
            resulting_state = self.magic_squares[resulting_state]
        reward = -1 if not self.is_terminal_state(resulting_state) else 0
        if not self.off_grid_move(resulting_state, self.agent_position):
            self.set_state(resulting_state)
            return resulting_state, reward, self.is_terminal_state(self.agent_position), None
        return self.agent_position, reward, self.is_terminal_state(self.agent_position), None

    def reset(self):
        self.agent_position = 0
        self.grid = np.zeros((self.m, self.n))
        self.add_magic_squares()
        self.grid[0][0] = 1  # mark the agent at the start square
        return self.agent_position

    def render(self):
        symbols = {0: '-', 1: 'X', 2: 'Ain', 3: 'Aout', 4: 'Bin', 5: 'Bout'}
        print('-' * 40)
        for row in self.grid:
            print('\t'.join(symbols[int(c)] for c in row), '\n')
        print('-' * 40)

```

How the tricky parts work

- **Magic squares on the grid.** Each teleporter occupies two squares (entrance and exit) drawn with values 2/3 and 4/5, so render() can show where each entrance and exit is. Row = state // m, column = state % n.
- **Terminal test.** Because S^+ minus S is exactly the terminal state, is_terminal_state is one set-membership check.
- **Illegal moves (off_grid_move).** A move is off-grid if the new square is outside S^+ , or if it would wrap around a row edge: moving left from column 0 lands on column m-1 of the previous row, and moving right from column m-1 lands on column 0 of the next row. The video leaves the reason as “an exercise for the reader” – this is it.
- **Step.** New square = old square + the action’s offset. If it is a teleporter entrance, replace it with the exit. Reward -1 unless the resulting square is terminal. Only legal moves change the state; illegal ones still cost -1.
- **Gym-style return.** step returns (state, reward, done, info); reset puts the agent at 0 and rebuilds the grid.

WATCH OUT Teleporters are applied *before* the legality check, so entering a magic square is always legal. If you add your own squares, keep entrances away from the edges or test the wrap-around logic.

9

Q-learning agent on Grid World

Video: 2:55 – 3:08 · Part II · Tabular methods

Q-learning main loop (tested)

```
def max_action(Q, state, actions):
    values = np.array([Q[state, a] for a in actions])
    return actions[np.argmax(values)] # numpy returns the first of any ties

if __name__ == '__main__':
    magic = {18: 54, 63: 14}
    env = GridWorld(9, 9, magic)
    ALPHA, GAMMA, EPS = 0.1, 1.0, 1.0 # learning rate, discount, exploration
    n_games = 50000
    Q = {(s, a): 0 for s in env.state_space_plus for a in env.possible_actions} # optimistic: all zeros
    total_rewards = np.zeros(n_games)
    for i in range(n_games):
        done, ep_rewards = False, 0
        obs = env.reset()
        while not done:
            if np.random.random() < 1 - EPS:
                a = max_action(Q, obs, env.possible_actions) # greedy
            else:
                a = np.random.choice(env.possible_actions) # random
            obs_, reward, done, info = env.step(a)
            ep_rewards += reward
            a_ = max_action(Q, obs_, env.possible_actions) # best action in the NEW state
            Q[obs, a] += ALPHA * (reward + GAMMA * Q[obs_, a_] - Q[obs, a])
            obs = obs_
        EPS = EPS - 2 / n_games if EPS - 2 / n_games > 0 else 0 # linear decay, 0 at half-way
        total_rewards[i] = ep_rewards
```

Hyper-parameters used in the video

α	γ	ϵ	Q initial values	Games	ϵ schedule
0.1	1.0 (totally far-sighted)	starts at 1.0	0 – “optimistic”	50,000	subtract 2 / n_games each game → purely greedy by game 25,000

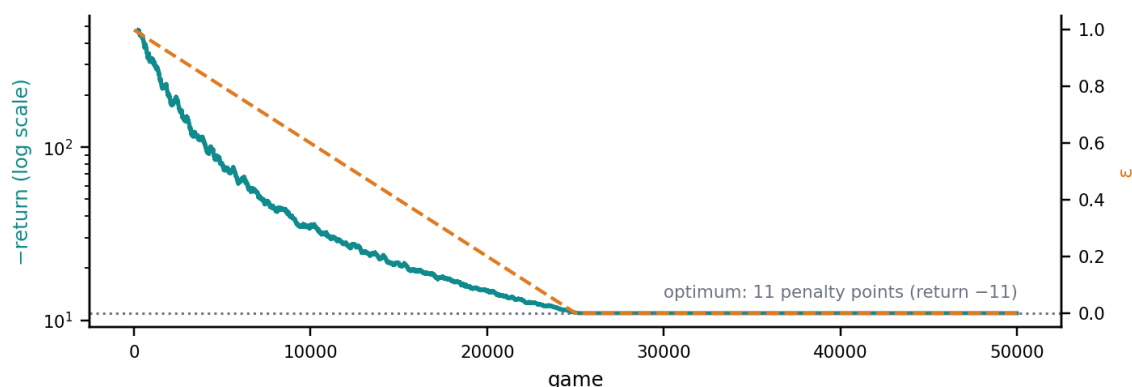


Figure 4 – Reproduced by running the code above (50,000 games, ≈ 14 s). Running mean of the penalty ($-$ return) on a log scale, and the ϵ schedule. The video’s own plot shows the same pattern.

Reading the result

- **The best possible return is -11 .** The route is 12 moves: square 0 → 9 → 18 (teleport to 54) → 55 → 64 → 73 → 74 ... 80. Eleven moves cost -1 each and the final move into the exit costs 0. In the video the converged score printed as ≈ -10.97 – “close enough for government work”.
- **It learns the shortcut.** Early episodes wander (the average penalty in the first 1,000 games is in the hundreds); by about game 25,000 – when ϵ hits 0 – every episode is optimal.
- **My reproduction:** the last 25,000 games all scored exactly -11 , and the greedy path matches the route above.

KEY IDEA Why optimistic initial values work here: every real reward is ≤ 0 , so a Q-table full of zeros is “too good to be true”. Each action the agent tries disappoints it, pushing it toward actions it has not tried yet – exploration for free. The environment’s `env.render()` can be called at the end to watch the agent’s route.

10

SARSA

Video: 3:29 – 3:32 · Part II · Tabular methods

SARSA is Q-learning's close cousin: both are **temporal-difference** algorithms (learn at every step), **model-free** (no transition probabilities needed) and **bootstrapped** (estimates built from estimates). The difference: **SARSA is on-policy, Q-learning is off-policy**. The name is the tuple it uses: **S**tate, **A**ction, **R**eward, next **S**tate, next **A**ction.

The update

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Compare with Q-learning: instead of the *maximum* over next actions, SARSA uses $Q(s', a')$ for the action a' the agent **actually chooses** next (via ϵ -greedy). It therefore learns the value of the policy it is really following – including its exploratory mistakes.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

(Q-learning, for comparison – note the max.)

Algorithm as shown on the slide

Algo

```

Initialize alpha (learning rate)
Initialize Q(s,a)
Initialize S
Choose A(S) using epsilon greedy from Q
Loop
  • Take action A, get reward, S'
  • Choose A'(S') using epsilon greedy from Q
  • Q(s,a) ← Q(s,a) + alpha[R + gamma*Q(s',a') - Q(s,a)]
  • S ← S', A ← A'

```

- Initialise α (learning rate) and $Q(s,a)$; initialise S .
- Choose A from S using ϵ -greedy on Q .
- Loop for each step of the episode:
- Take action A ; observe reward R and new state S' .
- Choose A' from S' using ϵ -greedy on Q .
- $Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma Q(S',A') - Q(S,A)]$.
- $S \leftarrow S', A \leftarrow A'$.

Important detail: A' is chosen *before* the update and then *reused* as the next action – you must not draw a second random action.

Frame from the video – the SARSA algorithm slide.

Q-learning vs. SARSA

	Q-learning	SARSA
Policy type	Off-policy (behaves ϵ -greedy, learns greedy)	On-policy (learns what it does)
Next-state term in the target	$\max_{a'} Q(s', a')$	$Q(s', a')$ for the sampled a'
Needs next action before update?	No	Yes
Effect of exploration noise	Ignored in the target	Included – the agent “knows” it might blunder
Terminal / initial values	Arbitrary except terminal = 0	Same

BEYOND THE VIDEO The textbook illustration is **cliff walking** (Sutton & Barto): next to a cliff, Q-learning learns the shortest route along the edge but occasionally falls off while exploring; SARSA, aware of its own exploration, learns a longer but safer route. As $\epsilon \rightarrow 0$ the two behave the same.

FROM THE VIDEO The instructor mentions that this material is a teaser for his longer course and invites viewers to speculate about results. Next page: SARSA applied to **CartPole**, a problem with a continuous state – which forces a trick.

11

SARSA on CartPole

Video: 3:32 – 3:40 · Part II · Tabular methods

CartPole: a cart slides along a track and must keep a pole upright. Two actions (push left / right), reward **+1 for every step the pole stays up**. The gym version stops at **200 steps**, so ≈ 200 is a near-perfect episode. The observation is four **continuous** numbers – cart position, cart velocity, pole angle, pole angular velocity.

The problem: a table needs discrete states

A Q-table cannot index a real number. The trick used here (instead of a neural network): cut each variable's range into ~ 10 equal buckets with `np.linspace` and map each reading to its bucket with `np.digitize`. Limits come from the environment documentation. Four variables $\times 10$ buckets $\Rightarrow \approx 10^4$ discrete states, and the Q-table has two columns (left, right).

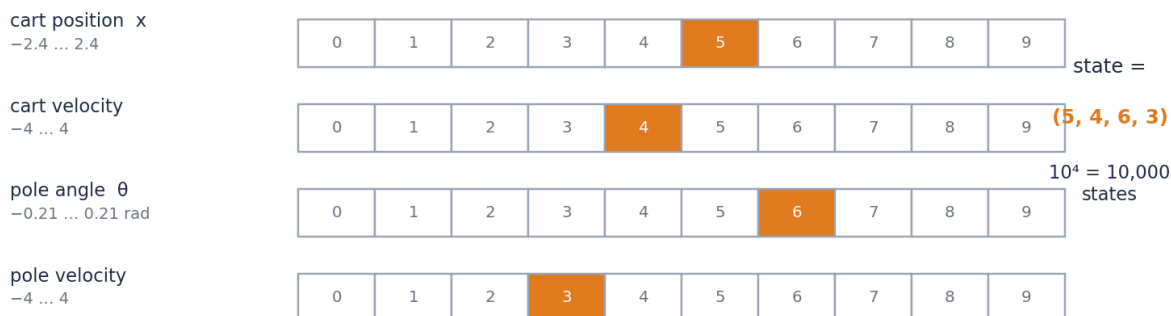


Figure 5 – Discretising the 4-number observation into a tuple of bucket indices (ranges as in the code below; the example reading is made up).

SARSA on CartPole (cleaned-up reconstruction of the tutorial's code; classic gym API)

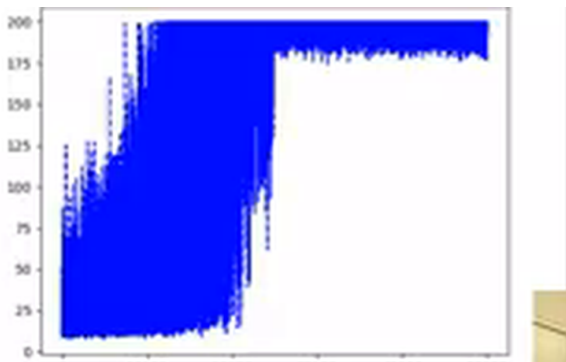
```
import gym, numpy as np
env = gym.make('CartPole-v0')
bins = [np.linspace(-2.4, 2.4, 10),          # classic API: reset() -> obs, step() -> 4 values
        np.linspace(-4, 4, 10),              # cart position (limits from the docs)
        np.linspace(-0.2094, 0.2094, 10),    # cart velocity (author's choice)
        np.linspace(-0.2094, 0.2094, 10),    # pole angle (rad)
        np.linspace(-4, 4, 10)]              # pole angular velocity

def get_state(obs):
    return tuple(int(np.digitize(o, b)) for o, b in zip(obs, bins))  # continuous 4-vector -> tuple of bucket indices

def max_action(Q, s, actions=(0, 1)):
    return int(np.argmax([Q[s, a] for a in actions]))

states = [(i, j, k, l) for i in range(11) for j in range(11) for k in range(11) for l in range(11)]
Q = {(s, a): 0.0 for s in states for a in (0, 1)}  # everything 0 (terminal must be 0)
ALPHA, GAMMA, EPS, n_games = 0.1, 0.9, 1.0, 50000
total_rewards = np.zeros(n_games)

for i in range(n_games):
    s = get_state(env.reset()); done = False; ep = 0
    a = max_action(Q, s) if np.random.random() < 1 - EPS else env.action_space.sample()
    while not done:
        obs_, r, done, info = env.step(a)
        s_ = get_state(obs_); ep += r
        a_ = max_action(Q, s_) if np.random.random() < 1 - EPS else env.action_space.sample()  # choose A' now
        Q[s, a] += ALPHA * (r + GAMMA * Q[s_, a_] - Q[s, a])  # SARSA update
        s, a = s_, a_  # reuse A'
    EPS = EPS - 2 / n_games if EPS > 0.01 else 0
    total_rewards[i] = ep
```



Frame from the video – scores over 50,000 SARSA games.

What the plot shows

- A messy blue band (every game plotted) with a clear **upward trend**.
- Once ϵ reaches its minimum the agent hits the **200-step cap most of the time**.
- Still plenty of variability: it does not balance for all 200 steps every game. The instructor's guess: the discretisation is too coarse – $\sim 10^4$ states cannot capture everything (e.g. combinations of velocity and position).
- Whole experiment ≈ 170 lines of code.

BEYOND THE VIDEO Counting buckets: `np.digitize` against 10 edges returns indices 0 ... 10 (11 values, including “below the first edge” and “above the last”), so the table can actually hold $11^4 = 14,641$ states – the code above uses `range(11)`. **Newer gym / gymnasium:** `reset()` returns (obs, info) and `step()` returns five values (terminated, truncated); adjust accordingly.

12

Maximization bias & Double Q-learning

Video: 3:40 – 3:43 · Part II · Tabular methods

Recap of Q-learning in the instructor's words: **model-free** (learns by playing, no transition dynamics needed), **bootstrapped** (starts from arbitrary estimates, except terminal states = 0) and **off-policy** (separate behaviour and target policies). It has one subtle flaw.

The problem: maximization bias

Suppose a state has many actions whose *true* value is 0. The agent's estimates $Q(s,a)$ are noisy – they spread above and below 0. Take the **max** over those noisy estimates and you almost always get something above 0. The true max is 0 but the max of the estimates is positive: a **positive bias**. It arises because the *same samples* are used both to pick the maximising action and to evaluate its value.

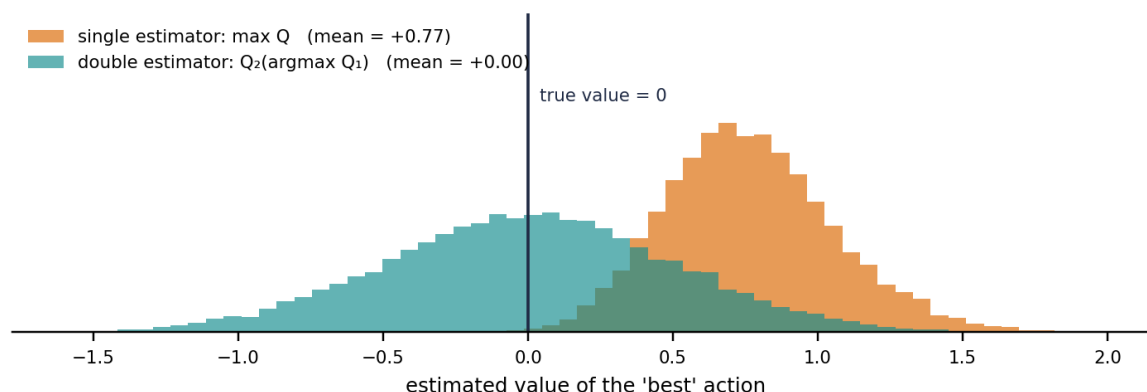


Figure 6 – Simulation (my own, beyond the video): 10 actions, all with true value 0, each estimated from 4 noisy samples. Max over one set of estimates is biased (+0.77); choosing the action with one estimator and evaluating it with an independent one is unbiased.

The fix: two Q-functions

Keep Q_1 and Q_2 . Use one to determine the maximising action and the *other* to determine the value of that action; alternate (randomly, with probability 0.5) which one is updated. Over time the bias is eliminated.

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha [r + \gamma Q_2(s', \arg\max_{a'} Q_1(s', a')) - Q_1(s, a)]$$

(When Q_2 is the one being updated, swap the subscripts.)

Double Q Learning

- Initialize alpha and epsilon
- Initialize $Q_1(s,a)$ and $Q_2(s,a)$
- Loop over episodes
 - Initialize S
 - For each episode
 - Choose A from S using epsilon greedy in $Q_1 + Q_2$
 - Take action A , observe R, S'
 - With 0.5 probability
 - $Q_1 \leftarrow Q_1 + \alpha [r + \gamma Q_2(S', \arg\max_{a'} Q_1(S', a)) - Q_1]$
 - $Q_2 \leftarrow Q_2 + \alpha [r + \gamma Q_1(S', \arg\max_{a'} Q_2(S', a)) - Q_2]$
 - $S \leftarrow S'$

Frame from the video – the Double Q-learning slide.

Algorithm

- Initialise α, ϵ ; initialise $Q_1(s,a)$ and $Q_2(s,a)$ (arbitrary, terminal = 0).
- For each episode, initialise S .
- For each step: choose A from S using ϵ -greedy on $Q_1 + Q_2$.
- Take A , observe R and S' .
- With probability 0.5 update Q_1 (using Q_2 's value of Q_1 's argmax); otherwise update Q_2 symmetrically.
- $S \leftarrow S'$. Repeat until done.

Combining two estimates for acting: you could take the max, the sum or the average – the course uses the **sum** of Q_1 and Q_2 .

KEY IDEA Double Q-learning removes the optimism of “max of noisy numbers”, but **at the cost of slower convergence** – you need to let it run longer (the course uses 100,000 games on CartPole versus 50,000 for SARSA).

13

Double Q-learning in code

Video: 3:43 – 3:54 · Part II · Tabular methods

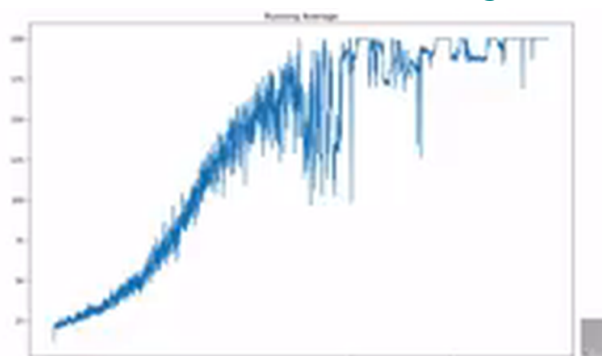
The script reuses everything from the SARSA tutorial (bucketing with `np.digitize`, the 10^4 -state table, the running-average plot over the last 100 games) and changes only the learning rule. Two details the instructor points out: NumPy's `argmax` returns the **first** element on ties, so identical Q-values for left and right will always pick “left”; and the Q-values for terminal states must be 0 – everything else can start anywhere.

Double Q-learning on CartPole (cleaned-up reconstruction)

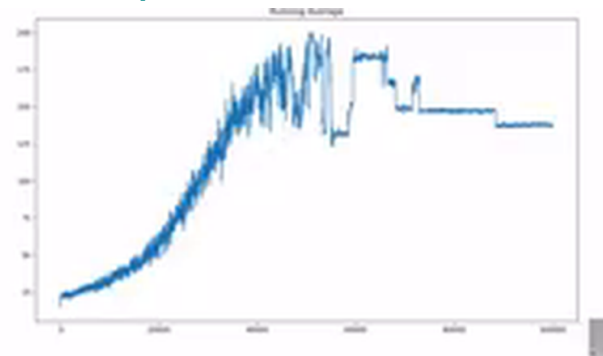
```
def max_action(Q1, Q2, s, actions=(0, 1)):
    return int(np.argmax([Q1[s, a] + Q2[s, a] for a in actions])) # act on the SUM of both tables

Q1 = {(s, a): 0.0 for s in states for a in (0, 1)} # states / get_state() as in the SARSA listing
Q2 = {(s, a): 0.0 for s in states for a in (0, 1)}
ALPHA, GAMMA, EPS, n_games = 0.1, 1.0, 1.0, 100000 # video compares GAMMA = 1.0 and 0.9

for i in range(n_games):
    s = get_state(env.reset()); done = False; ep = 0
    while not done:
        a = max_action(Q1, Q2, s) if np.random.random() < 1 - EPS else env.action_space.sample()
        obs_, r, done, info = env.step(a)
        s_ = get_state(obs_); ep += r
        if np.random.random() < 0.5: # coin flip: which table to update?
            a_ = max_action(Q1, Q1, s_) # action chosen by Q1 ...
            Q1[s, a] += ALPHA * (r + GAMMA * Q2[s_, a_] - Q1[s, a]) # ... valued by Q2
        else:
            a_ = max_action(Q2, Q2, s_) # action chosen by Q2 ...
            Q2[s, a] += ALPHA * (r + GAMMA * Q1[s_, a_] - Q2[s, a]) # ... valued by Q1
        s = s_
    EPS = EPS - 2 / n_games if EPS > 0.01 else 0
```

Experiments in the video: the same algorithm, two values of γ 

$\gamma = 1.0$ – running average of the last 100 games over 100,000 games.



$\gamma = 0.9$ – the same code, a different outcome.

- **$\gamma = 1.0$:** the agent starts learning quickly and, by about 60,000 games, **consistently reaches the 200-move cap**.
- **$\gamma = 0.9$:** it touches 200 for a handful of games, then stutters and **gets worse**. The instructor admits he does not know exactly why: an earlier recording with $\gamma = 0.9$ worked fine, so random-number variation or uneven coverage of the state space may be to blame.
- **Why the difference might make sense:** the transitions are deterministic, so discounting is not needed for uncertainty, and $\gamma = 0.9$ shrinks the value of rewards far in the future (see the “ ≈ 10 ” calculation in Section 3).

WATCH OUT Take-aways: (1) hyper-parameters matter a lot; (2) **run several seeds** before concluding anything – a single curve can be misleading; (3) Double-Q trades convergence speed for unbiased estimates, so give it more episodes.

BEYOND THE VIDEO The discretised terminal observation can fall in the same bucket as a non-terminal state, so a safer implementation treats `done` explicitly: use `target = r` when the episode has ended instead of bootstrapping from $Q(s', a')$.



Checkpoint II - Tabular methods

Video: recap of 2:34 – 3:08 and 3:29 – 3:54 · Part II

	Q-learning	SARSA	Double Q-learning
Update target	$r + \gamma \max_{a'} Q(s', a')$	$r + \gamma Q(s', a')$ with the a' actually chosen	$r + \gamma Q_2(s', \arg\max Q_1(s', \cdot))$ (and swap)
Policy type	off-policy	on-policy	off-policy
Tables	1	1	2 (act on the sum)
Fixes / adds	baseline TD control	accounts for own exploration	removes maximization bias
Cost	positive bias from max	needs a' before the update	slower to converge → more games
Used in video on	Grid World (50,000 games, -11)	CartPole (50,000 games)	CartPole (100,000 games)
Typical α, γ	0.1, 1.0	0.1, 0.9	0.1, 1.0 vs 0.9

Debug checklist for tabular agents

- **Terminal Q = 0?** Initialise anywhere you like, but terminal states must be 0 (or handle done explicitly).
- **Reset every episode:** done flag, per-episode reward and the environment; keep ϵ changing *between* episodes.
- **Are you updating the right pair?** $Q[s, a]$ uses the *old* state s and the action you took – update before you overwrite $s = s'$.
- **Illegal moves:** in Grid World, off-grid moves must cost -1 and leave the state unchanged, or the agent will “discover” wrap-around exploits.
- **Ties:** $\arg\max$ returns the first maximum, which biases early behaviour when all values are equal.
- **Print progress** every few thousand games so you can tell “slow” from “broken”.

What the experiments teach

Experiment	Observation	Lesson
Grid World + Q-learning	Return climbs from the hundreds negative to exactly -11 as $\epsilon \rightarrow 0$	Tabular Q-learning finds the optimum and the shortcut when exploration decays
CartPole + SARSA	Trend up, reaches ≈ 200 , but noisy	Coarse discretisation limits what the table can represent
CartPole + Double Q, $\gamma = 1.0$	Consistently ≈ 200 after $\approx 60k$ games	Unbiased, but slow – give it more games
CartPole + Double Q, $\gamma = 0.9$	Peaks then degrades	Hyper-parameters and random seeds matter; repeat runs

KEY IDEA Limits of tables. Everything in Part II needs a state space small enough to enumerate (or crush into buckets). Breakout and Space Invaders have screens of 210×160 pixels – no table could hold them. That is the motivation for Part III.

Mini-project

Port the Q-learning agent to a **5 × 5 Grid World** with one teleporter. (1) Predict the optimal return by hand. (2) Train and compare with your prediction. (3) Swap Q-learning for SARSA and Double Q-learning and plot all three learning curves on the same axes. Does anything change when you make the reward stochastic (e.g. $+1$ or -3 with equal probability for one square)? That is where Double Q-learning should help.

14

From tables to neural networks: DQN overview

Video: 0:01 – 0:08 and 0:52 – 0:53 · Part III · Deep Q-learning

When the state space is huge (an Atari screen) or continuous, a table is impossible. **Deep Q-learning** replaces the table with a **deep neural network** that takes an observation and outputs one value per action. The justification given in the video: neural networks are **universal function approximators** – they can approximate any continuous function, and the relationship between states, actions and future returns is exactly such a function.

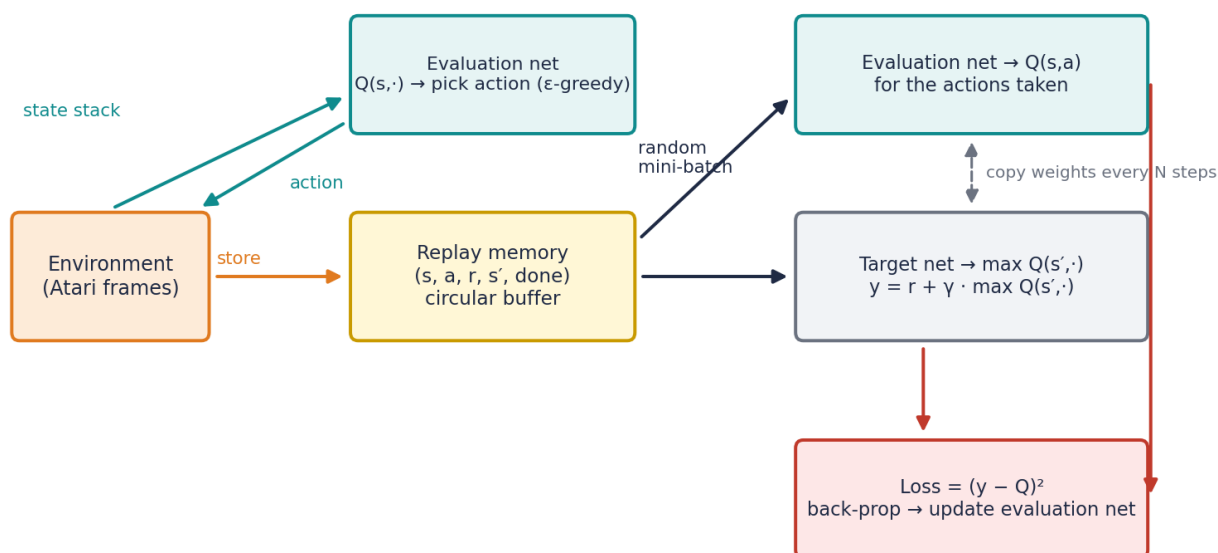


Figure 7 – One training step of DQN. Redrawn from the instructor's verbal description; the on-screen captions in the video read "Two networks reduce bias", "Loss = Q(actual) – Q(maximal)" and "Stack new frame and store frames, action, reward in memory".

The recipe, step by step

- **Memory.** The agent remembers the states it saw, the actions it took and the rewards it got (plus the next state and a terminal flag).
- **Learning step.** Sample a *random* subset of the memory; feed the states through the network and compute the values of the actions that were taken. As in tabular Q-learning, also compute the value of the *maximal* action; the difference between the two is the **loss** used to update the network weights.
- **Two networks.** An **evaluation network** evaluates the current state and decides which action to take; a **target network** calculates the value of the maximal action during learning. Using two networks eliminates bias in the action-value estimates (cf. maximization bias, Section 12). The target network's weights are periodically refreshed from the evaluation network.
- **Pixels ⇒ convolutions.** For image observations a convolutional network extracts features; its output is flattened and passed to dense layers which output the action values.
- **Motion ⇒ frame stacking.** A single image cannot tell you which way the ball is moving. The CNN therefore receives a **stack of consecutive frames** (3–4) as one input.
- **Acting.** Draw a random number: if below ϵ , act randomly; otherwise pass the stacked frames through the evaluation network and take the arg-max action. Append the new frame to the stack and store the transition.

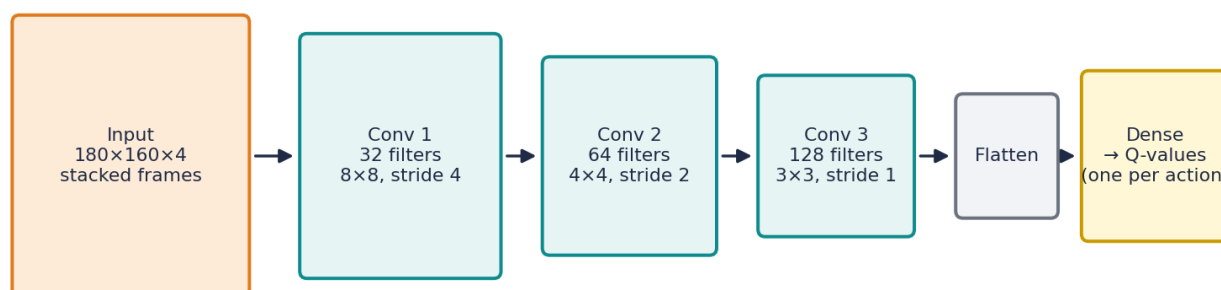


Figure 8 – The convolutional Q-network used for Breakout in the TensorFlow tutorial (layer sizes from the video).

FROM THE VIDEO "That's really all there is to deep Q-learning: a couple of neural networks, a memory to keep track of states, and lots of GPU horsepower." – Both TensorFlow and PyTorch are fine, as long as the framework lets you use a GPU *and* save models (training can take a day or more).

WATCH OUT Deep Q-learning is **off-policy** and learns **every step** (from a replayed batch), unlike policy gradients which wait until an episode ends. It also needs a lot of RAM for the memory (Section 17).

15

DQN ingredients in detail

Video: 0:08 – 0:49 and 1:06 – 1:28 · Part III · Deep Q-learning

DQN pseudo-code

```

initialise evaluation net Q_e and target net Q_t (same architecture); replay memory D of size N
for each episode:
    s = stack_frames(preprocess(env.reset()))
    while not done:
        a = random action          if random() < epsilon
            argmax_a Q_e(s, a)      otherwise
        s', r, done = env.step(a); s' = stack_frames(preprocess(s'))
        D.store(s, a, r, s', done)  # overwrite the oldest entry when full
        batch = random, non-sequential sample of D
        y = r + gamma * max_a' Q_t(s', a') * (1 - done)  # y = r if s' is terminal
        loss = (y - Q_e(s, a))^2  -> gradient step on Q_e
        every K steps: copy Q_e weights into Q_t (optional - see Section 19)
        decay epsilon (floor 0.01 - 0.05)

```

$$y = r + \gamma \max_a Q_{\text{target}}(s', a') \quad (y = r \text{ if } s' \text{ is terminal})$$

$$L(\theta) = (y - Q_{\text{eval}}(s, a; \theta))^2$$

Replay memory

- Fixed size. A counter tracks the number of stored memories; the write index is counter % size, so when memory is full it **wraps to the start and overwrites the oldest data**.
- Store: state, action (one-hot in the TF version), reward, next state and a **terminal flag** (so the target can drop the future term at the end of an episode).
- Sample non-sequentially**. Consecutive transitions are strongly correlated; learning from them gets the agent “trapped in some little nook and cranny of parameter space”, giving oscillating performance. Random sampling across the whole memory gives robust learning.
- Optional trick used in both tutorials: **pre-fill the memory with random play** before learning starts so the first batches are not empty. The author calls it a hack but considers it legitimate.

Building the target

The network outputs a Q-value for every action but only one action was taken. The trick: **copy the network's own predictions into the target**, then overwrite just one entry with $r + \gamma \cdot \max Q(s')$. The loss for all other actions is then exactly zero, so only the taken action's value moves. For terminal transitions multiply the future term by 0 so the target is just r .

Pre-processing observations

Step	Why	Breakout (TF)	Space Invaders (PyTorch)
Crop	Score, borders and bottom rows carry little information; saves memory	rows 30 → end: 210×160 → 180×160	obs[15:200, 30:125] → 185×95
Gray-scale	Colour is irrelevant; “is an enemy there or not?”	mean over the 3 channels	mean over the 3 channels
Stack frames	A single frame has no motion information	4 frames: 180×160×4	3 frames (DeepMind used 4)
Frame repeat	gym Atari repeats each action for 2-4 random frames	stacking 4 covers it	repeat the action for 3 steps

Stacking logic: at the start of an episode fill the whole stack with the first frame; afterwards drop the oldest frame, shift the others down and append the newest (frames 1-2-3-4 → 2-3-4-5).

ε decay in these tutorials

Breakout: start fully random for $\approx 100,000$ moves, then multiply ϵ by a number just under 1 each step down to a floor of **0.01**. Space Invaders: wait a set number of steps, then decrease linearly to **0.05**. DeepMind used a linear schedule; any shape works.

BEYOND THE VIDEO Treat the frame stack as **channels** of one image (as in the Breakout network, input 180×160×4) – that is how the original DeepMind agent works. See the design note in Section 18 for how the PyTorch video differs.

16

DQN in TensorFlow (Breakout): the network

Video: 0:09 – 0:21 · Part III · Deep Q-learning

First DQN tutorial: teach an agent to play **Breakout** in TensorFlow. The code is split into two classes – one for the deep Q-network, one for the agent. Actions: move left, move right, fire the ball (3 actions). Atari frames are 210 × 160; after cropping and stacking four frames the network input is **180 × 160 × 4**.

TensorFlow 1.x style, as in the video (use `tf.compat.v1` on TF 2) – reconstructed from the narration

```
class DeepQNetwork(object):
    def __init__(self, lr, n_actions, name, fcl_dims=1024, input_dims=(180, 160, 4), chkpt_dir='tmp/dqn'):
        self.lr, self.n_actions, self.name = lr, n_actions, name
        self.fcl_dims, self.input_dims = fcl_dims, input_dims
        self.sess = tf.Session() # each network owns a session
        self.build_network()
        self.sess.run(tf.global_variables_initializer()) # TF1: forgetting this raises an error
        self.saver = tf.train.Saver() # checkpoints: training takes hours
        self.checkpoint_file = os.path.join(chkpt_dir, 'deepqnet.ckpt')
        self.params = tf.get_collection(tf.GraphKeys.TRAINABLE_VARIABLES, scope=self.name) # for copying later

    def build_network(self):
        with tf.variable_scope(self.name): # scope = network name (q_eval / q_next)
            self.input = tf.placeholder(tf.float32, shape=[None, *self.input_dims], name='inputs')
            self.actions = tf.placeholder(tf.float32, shape=[None, self.n_actions], name='action_taken')
            self.q_target = tf.placeholder(tf.float32, shape=[None, self.n_actions], name='q_target')
            init = tf.variance_scaling_initializer(scale=2) # same initialiser as the DeepMind paper
            conv1 = tf.layers.conv2d(self.input, 32, (8, 8), strides=4, name='conv1', kernel_initializer=init)
            conv1_a = tf.nn.relu(conv1)
            conv2 = tf.layers.conv2d(conv1_a, 64, (4, 4), strides=2, name='conv2', kernel_initializer=init)
            conv2_a = tf.nn.relu(conv2)
            conv3 = tf.layers.conv2d(conv2_a, 128, (3, 3), strides=1, name='conv3', kernel_initializer=init)
            conv3_a = tf.nn.relu(conv3)
            flat = tf.layers.flatten(conv3_a)
            dense1 = tf.layers.dense(flat, self.fcl_dims, activation=tf.nn.relu, kernel_initializer=init)
            self.Q_values = tf.layers.dense(dense1, self.n_actions, kernel_initializer=init) # linear: raw Q per action
            self.q = tf.reduce_sum(tf.multiply(self.Q_values, self.actions), axis=1) # Q of the action taken
            self.loss = tf.reduce_mean(tf.square(self.Q_values - self.q_target)) # squared difference
            self.train_op = tf.train.AdamOptimizer(self.lr).minimize(self.loss)

    def load_checkpoint(self): self.saver.restore(self.sess, self.checkpoint_file)
    def save_checkpoint(self): self.saver.save(self.sess, self.checkpoint_file)
```

Design choices explained

- **Two instances, one class.** The name argument creates a separate variable scope, so “q_eval” and “q_next” are two independent networks, each with its own TensorFlow session and checkpoint folder.
- **Placeholders with None first.** `shape=[None, ...]` means “batch size unknown”; you always train on batches. Naming placeholders and layers makes TensorFlow errors point to the right place.
- **Filters 32 / 64 / 128** with kernels 8×8 / 4×4 / 3×3 and strides 4 / 2 / 1 – the layer pattern from the DeepMind paper, each followed by ReLU; the same variance-scaling initialiser (scale 2) is used everywhere. “Learn from the experts.”
- **Linear output.** The last layer is not activated: Q-values are unbounded real numbers, one per action.
- **Loss.** The squared difference between the network’s Q-values and `q_target` (built in Section 17). Optimiser: Adam. This is a form of temporal-difference learning – each step it compares the value of the action taken with the maximal action.
- **Checkpoints.** The network takes “forever” to train (the author ran ≈ 10 hours for a score of 2–3), so save often – every 10 games – and be able to reload.
- **Trainable variables.** Collecting params per network lets us copy one network’s weights into the other.

WATCH OUT Check the learning-rate and layer numbers against the author’s GitHub (linked in the video description) before reproducing results: they are read from auto-captions and a few digits are ambiguous. Learning rate in this tutorial: **0.00025**.

17

DQN in TensorFlow (Breakout): the agent

Video: 0:21 – 0:49 · Part III · Deep Q-learning

agent.py - reconstructed from the narration (TF 1.x style)

```

class Agent(object):
    def __init__(self, alpha, gamma, mem_size, n_actions, epsilon, batch_size, replace_target=30000,
                 input_dims=(180, 160, 4), q_next_dir='tmp/q_next', q_eval_dir='tmp/q_eval'):
        self.n_actions, self.action_space = n_actions, [i for i in range(n_actions)]
        self.gamma, self.mem_size, self.epsilon = gamma, mem_size, epsilon
        self.batch_size, self.replace_target, self.mem_cntr = batch_size, replace_target, 0
        self.q_eval = DeepQNetwork(alpha, n_actions, input_dims=input_dims, name='q_eval', chkpt_dir=q_eval_dir)
        self.q_next = DeepQNetwork(alpha, n_actions, input_dims=input_dims, name='q_next', chkpt_dir=q_next_dir)
        self.state_memory = np.zeros((mem_size, *input_dims))
        self.new_state_memory = np.zeros((mem_size, *input_dims))
        self.action_memory = np.zeros((mem_size, n_actions), dtype=np.int8) # one-hot actions
        self.reward_memory = np.zeros(mem_size)
        self.terminal_memory = np.zeros(mem_size, dtype=np.int8) # stores 1 - done

    def store_transition(self, state, action, reward, state_, terminal):
        index = self.mem_cntr % self.mem_size # wraps around: overwrite the oldest
        self.state_memory[index], self.new_state_memory[index] = state, state_
        actions = np.zeros(self.n_actions); actions[action] = 1.0
        self.action_memory[index], self.reward_memory[index] = actions, reward
        self.terminal_memory[index] = 1 - terminal
        self.mem_cntr += 1

    def choose_action(self, state):
        if np.random.random() < self.epsilon:
            return np.random.choice(self.action_space)
        q = self.q_eval.sess.run(self.q_eval.Q_values, feed_dict={self.q_eval.input: state[np.newaxis, :]})
        return np.argmax(q)

    def learn(self):
        if self.mem_cntr % self.replace_target == 0:
            self.update_graph() # eval -> target
        batch = np.random.choice(min(self.mem_cntr, self.mem_size), self.batch_size) # random, non-sequential
        state_batch, new_state_batch = self.state_memory[batch], self.new_state_memory[batch]
        action_batch = self.action_memory[batch]
        action_indices = np.dot(action_batch, np.array([0, 1, 2], dtype=np.int8)) # one-hot -> integer
        reward_batch, terminal_batch = self.reward_memory[batch], self.terminal_memory[batch]
        q_eval = self.q_eval.sess.run(self.q_eval.Q_values, feed_dict={self.q_eval.input: state_batch})
        q_next = self.q_next.sess.run(self.q_next.Q_values, feed_dict={self.q_next.input: new_state_batch})
        q_target = q_eval.copy() # loss = 0 for actions not taken
        idx = np.arange(self.batch_size)
        q_target[idx, action_indices] = reward_batch + self.gamma * np.max(q_next, axis=1) * terminal_batch
        self.q_eval.sess.run(self.q_eval.train_op, feed_dict={self.q_eval.input: state_batch, self.q_eval.q_target: q_target})
        if self.mem_cntr > 100000: # first 100k steps: epsilon stays at 1.0
            self.epsilon = max(self.epsilon * 0.9999999, 0.01)

    def update_graph(self):
        for t, e in zip(self.q_next.params, self.q_eval.params):
            self.q_eval.sess.run(tf.assign(t, e)) # use the session of the network we copy FROM

```

Main program settings

- Environment: **Breakout**; epsilon starts at **1.0**; learning rate $\alpha \approx 0.00025$.
- Input $180 \times 160 \times 4$ (four stacked frames); 3 actions.
- Memory **25,000** transitions; batch size **32**; 200 games; save every 10 games.
- Memory is pre-filled with random play. Quirk: the agent's actions are 0-2 but the game's are 1-3, so add 1 before stepping and subtract 1 before storing.
- Learn at every step; append the score each episode and plot score (should slope up) and ϵ (should slope down).

Practical notes

- RAM:** 25,000 stored states $\approx$ **48 GB** on the author's PC (he has 96 GB). With 16 GB, cut the memory to about 6,000-7,000.
- Speed:** after $\approx$ 3,000 games ($\approx$ 10 h) it scored only 2-3 points per five lives; it needs days of GPU training to be competitive.
- Pitfall:** the copy operation needs the session of the *source* network - the wrong session raises a confusing error.
- Epsilon never goes below 0.01: always keep testing the model.

BEYOND THE VIDEO Why 48 GB? Two arrays (states and next-states) of $25,000 \times 180 \times 160 \times 4$ values stored as 64-bit floats $\approx$ 46 GB. Storing frames as uint8 (0-255) and normalising when sampling would cut this by a factor of 8 with no loss of information.

KEY IDEA Reading the learn() function in five lines: (1) maybe copy eval $\rightarrow$ target; (2) pick a random batch; (3) get $Q(s, \cdot)$ from the eval net and $Q(s', \cdot)$ from the target net; (4) copy $Q(s, \cdot)$ and replace the entry of the taken action by $r + \gamma \cdot \max Q(s', \cdot) \cdot (1 - \text{done})$; (5) one gradient step so $Q(s, a)$ moves toward that target; then shrink ϵ .

18

DQN in PyTorch (Space Invaders): network & agent

Video: 0:52 – 1:15 · Part III · Deep Q-learning

Three-part series: (1) the CNN class, (2) the agent class, (3) the main loop. PyTorch is praised as “highly expressive” with little boilerplate; TensorFlow remains the choice for cross-platform deployment. A GPU is essential – on CPU it would take “7,000 years”.

model.py / agent - reconstructed from the narration (PyTorch)

```
class DeepQNetwork(nn.Module):
    def __init__(self, ALPHA):
        super(DeepQNetwork, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 8, stride=4, padding=1) # 1 channel: grayscale
        self.conv2 = nn.Conv2d(32, 64, 4, stride=2)
        self.conv3 = nn.Conv2d(64, 128, 3)
        self.fc1 = nn.Linear(128 * 19 * 8, 512) # 19 x 8 found by running the code
        self.fc2 = nn.Linear(512, 6) # 6 actions in Space Invaders
        self.optimizer = optim.RMSprop(self.parameters(), lr=ALPHA)
        self.loss = nn.MSELoss()
        self.device = T.device('cuda:0' if T.cuda.is_available() else 'cpu')
        self.to(self.device)

    def forward(self, observation):
        observation = T.Tensor(np.array(observation)).to(self.device) # frames -> tensor on the GPU
        observation = observation.view(-1, 1, 185, 95) # channels first
        observation = F.relu(self.conv1(observation))
        observation = F.relu(self.conv2(observation))
        observation = F.relu(self.conv3(observation))
        observation = observation.view(-1, 128 * 19 * 8) # flatten
        observation = F.relu(self.fc1(observation))
        return self.fc2(observation) # Q-values (one row per frame)

class Agent(object):
    def __init__(self, gamma, epsilon, alpha, maxMemorySize, epsEnd=0.05, replace=10000, actionSpace=[0, 1, 2, 3, 4, 5]):
        self.GAMMA, self.EPSILON, self.EPS_END = gamma, epsilon, epsEnd
        self.actionSpace, self.memSize, self.steps = actionSpace, maxMemorySize, 0
        self.learn_step_counter, self.memory, self.memCntr = 0, [], 0 # a LIST of [s, a, r, s'] (cheaper than stacking arrays)
        self.replace_target_cnt = replace
        self.Q_eval = DeepQNetwork(alpha) # estimate of current states
        self.Q_next = DeepQNetwork(alpha) # estimate of successor states

    def storeTransition(self, state, action, reward, state_):
        if self.memCntr < self.memSize:
            self.memory.append([state, action, reward, state_])
        else:
            self.memory[self.memCntr % self.memSize] = [state, action, reward, state_]
            self.memCntr += 1

    def chooseAction(self, observation):
        rand = np.random.random()
        actions = self.Q_eval.forward(observation) # matrix: frames x 6 actions
        if rand < 1 - self.EPSILON:
            action = T.argmax(actions[1]).item() # greedy
        else:
            action = np.random.choice(self.actionSpace) # explore
        self.steps += 1
        return action
```

Why these numbers?

- **Space Invaders' 6 actions** (a subset of the 18 in the Atari library): do nothing, move left, move right, fire, move left + fire, move right + fire.
- **128 × 19 × 8**: a 185 × 95 gray image through the three convolutions leaves 128 feature maps of 19 × 8 – found “by trial and error”, but you can compute it: conv1 (k8, s4, p1) → 45×23, conv2 (k4, s2) → 21×10, conv3 (k3) → 19×8.
- **Loss and optimiser**: mean-squared error between the target and predicted Q-values; RMSprop. Q-learning is off-policy: the ϵ -greedy behaviour generates data, the purely greedy target policy is what is updated.
- **Memory as a list**: stacking NumPy arrays is “computationally prohibitive”; appending to a list and converting at learning time is much faster.
- **Target network**: coded for completeness (replace argument), but the author found it did *not* help here and leaves it off (replace=None) – results in Section 19.
- **ϵ -greedy form**: the test is $\text{rand} < 1 - \epsilon$ for the greedy branch, a “soft- ϵ ” variant – greedy action probability $\epsilon/6$ higher than ϵ alone.

BEYOND THE VIDEO Design note. This network treats each stacked frame as a separate one-channel image, hence “a matrix with one row per frame” and the odd actions[1] indexing. The Breakout network feeds the frames as channels, which is the usual DQN design. If you reimplement, use `in_channels = number of stacked frames` and one row of Q-values per sample.

19

DQN in PyTorch: learning, main loop & results

Video: 1:15 – 1:46 · Part III · Deep Q-learning

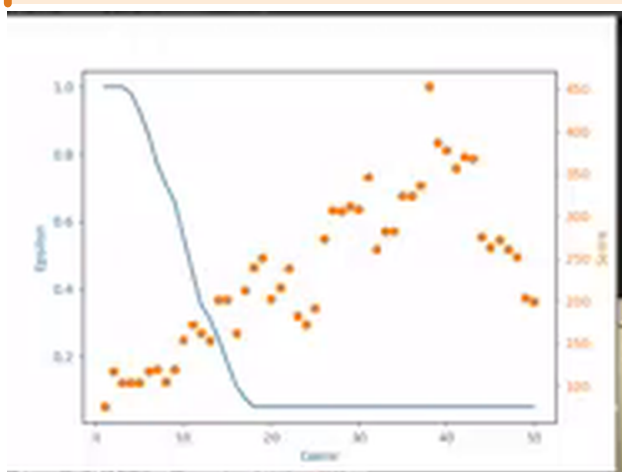
learn() - follows the video's logic

```
def learn(self, batch_size):
    self.Q_eval.optimizer.zero_grad() # PyTorch accumulates gradients: reset each batch
    if self.replace_target_cnt is not None and self.learn_step_counter % self.replace_target_cnt == 0:
        self.Q_next.load_state_dict(self.Q_eval.state_dict()) # eval -> target (optional)
    if self.memCntr + batch_size < self.memSize: # pick a random start that leaves room for a batch
        memStart = int(np.random.choice(range(self.memCntr)))
    else:
        memStart = int(np.random.choice(range(self.memSize - batch_size - 1)))
    miniBatch = self.memory[memStart:memStart + batch_size]
    memory = np.array(miniBatch, dtype=object)
    Qpred = self.Q_eval.forward(list(memory[:, 0][:])).to(self.Q_eval.device) # value of current states
    Qnext = self.Q_next.forward(list(memory[:, 3][:])).to(self.Q_eval.device) # value of successor states
    maxA = T.argmax(Qnext, dim=1).to(self.Q_eval.device) # greedy action
    rewards = T.Tensor(list(memory[:, 2])).to(self.Q_eval.device)
    Qtarget = Qpred.clone() # loss is zero everywhere except one entry
    Qtarget[:, maxA] = rewards + self.GAMMA * T.max(Qnext[1])
    if self.steps > 500: # linear epsilon decrease after a warm-up
        self.EPSILON = self.EPSILON - 1e-4 if self.EPSILON > self.EPS_END else self.EPS_END
    loss = self.Q_eval.loss(Qtarget, Qpred).to(self.Q_eval.device)
    loss.backward(); self.Q_eval.optimizer.step()
    self.learn_step_counter += 1
```

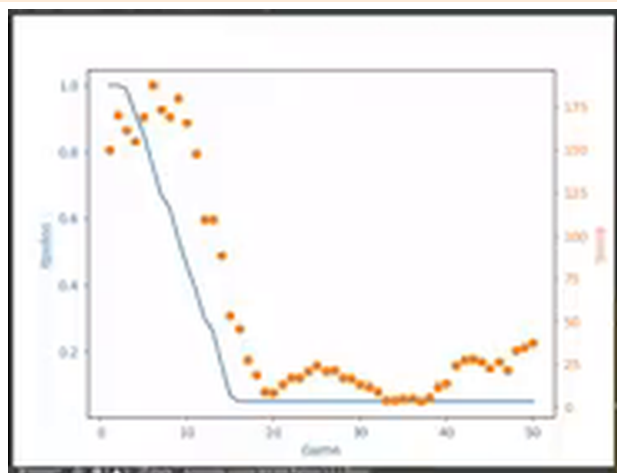
main loop (condensed)

```
env = gym.make('SpaceInvaders-v0')
brain = Agent(gamma=0.95, epsilon=1.0, alpha=0.03, maxMemorySize=5000, replace=None) # values as heard; check the repo
crop = lambda o: np.mean(o[15:200, 30:125], axis=2) # crop + gray-scale
while brain.memCntr < brain.memSize: # 1) fill memory with RANDOM play
    obs = env.reset(); done = False
    while not done:
        a = env.action_space.sample(); obs_, r, done, info = env.step(a)
        if done and info['ale.lives'] == 0: r = -100 # "losing really sucks" (size is a design choice)
        brain.storeTransition(crop(obs), a, r, crop(obs_)); obs = obs_
    scores, epsHistory = [], []
    for i in range(50): # 2) train for 50 games, batch size 32
        epsHistory.append(brain.EPSILON); done = False
        obs = env.reset(); frames = [crop(obs)]; score = 0; lastAction = 0
        while not done:
            if len(frames) == 3: action = brain.chooseAction(frames); frames = [] # act every 3rd step ...
            else: action = lastAction # ... repeat in between
            obs_, r, done, info = env.step(action); score += r; frames.append(crop(obs_))
            if done and info['ale.lives'] == 0: r = -100
            brain.storeTransition(crop(obs), action, r, crop(obs_)); obs = obs_
        brain.learn(32); lastAction = action
    scores.append(score)
```

WATCH OUT Two quirks to know about. (1) The mini-batch is a *contiguous slice* of memory from a random start - more correlated than true random sampling (Section 15 recommends `np.random.choice` over indices). (2) The target overwrites the entry at the **arg-max of Q_{next}** whereas standard DQN overwrites the entry of the action actually **taken** (as the Breakout TF agent does). It still learned in the author's runs; use the taken action in your own code.



No target replacement: ϵ falls linearly (blue) while the score (orange dots) climbs, peaking near 450.



With target replacement: the score collapses as ϵ falls.

Results and lessons

- Without target replacement, ϵ falls *linearly* and the 5-game running score rises “quite dramatically”; runs in excess of **700 points** were seen. A dip near the end happens often - the agent wanders between local minima and usually recovers if trained longer.
- With a target network the author repeatedly saw a **nose-dive** and oscillations (likely from the weight copies). He double-checked that the implementation just copies one state-dict into the other and could not find a bug - so he leaves it off. Experimentally, it is a hyper-parameter, not a given.
- Batch size 32** (≈ 30 minutes for 50 games on an i7-7820X, 32 GB RAM, GTX 1080 Ti). Larger batches help only slightly but more than double the run time.
- Plot ϵ and the average of the last 5 games together - the relation between exploration and score is the thing to watch.

20

Policy gradients: the idea and the math

Video: 1:46 – 1:50 · Part IV · Policy gradients

Value-based methods (Q-learning, DQN) learn how good states and actions are and derive a policy from that. **Policy-gradient methods learn the policy directly.** A policy is just the probability distribution the agent uses to pick actions, so we use a deep neural network to approximate it: the network takes an observation and outputs action probabilities through a **softmax**.

The algorithm (REINFORCE)

1. **Generate an episode**, sampling every action from the network's softmax output; store states, actions and rewards in memory.
2. At the episode's end, go back through the memory and compute the **discounted future return** G_t at every time step.
3. Use the returns as **weights** and the actions actually taken as **labels**; back-propagate to update the network's weights.
4. **Repeat** until the agent is good.

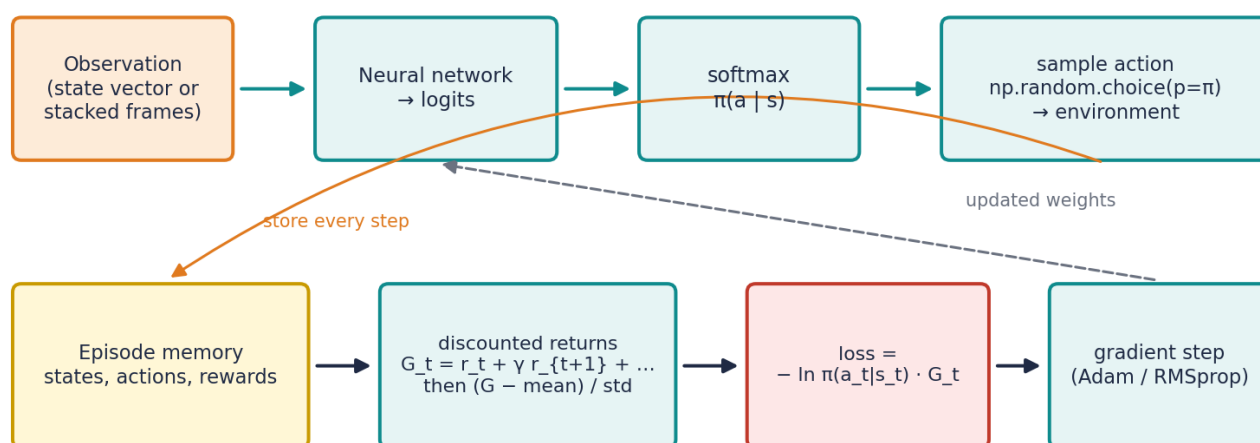


Figure 9 - The policy-gradient training loop (redrawn from the description in the video and the Lunar Lander code).

Where the update comes from (equations shown on screen)

Let $J(\theta)$ measure the agent's performance, a function of the network weights θ . Update with a learning rate α . Because we want performance to *increase*, this is gradient **ascent**, not descent:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

The policy-gradient theorem says the gradient is proportional to a sum over states (weighted by how much time is spent in each, $\mu(s)$) and over actions of the action values times the gradient of the policy:

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla_\theta \pi(a | s, \theta)$$

After some manipulation this is an expectation, and plugging it into the update gives the sample-based rule shown last in the video:

$$\theta_{t+1} = \theta_t + \alpha G_t \frac{\nabla_\theta \pi(A_t | S_t, \theta_t)}{\pi(A_t | S_t, \theta_t)} = \theta_t + \alpha G_t \nabla_\theta \ln \pi(A_t | S_t, \theta_t)$$

How to read the final equation

- G_t - the discounted future return after time t (Section 3).
- $\nabla \pi(A_t | S_t) / \pi(A_t | S_t)$ ($= \nabla \ln \pi$) - a vector pointing in the direction in parameter space that most increases the probability of repeating the action A_t .
- Multiply them: the update **increases the probability of actions that were followed by high returns** and (for negative G) decreases it for actions followed by poor outcomes. That is precisely how the agent learns.

KEY IDEA In code, "increase the probability of the action taken, weighted by G " becomes: **loss = $(-\ln \pi(a_t | s_t)) \times G_t$** . TensorFlow's `sparse_softmax_cross_entropy_with_logits` computes $-\ln \pi$ for you - which is why the network's last layer is left un-activated ("logits").

21

Why policy gradients? Variance & sample efficiency

Video: 1:50 – 1:55 · Part IV · Policy gradients

Why bother, given DQN?

- **The policy may be simpler than the value function.** Value-based agents must learn an accurate action-value function – often very complex. In some cases the optimal policy itself is much simpler to approximate, so a policy-gradient agent can beat an environment faster.
- **Deterministic optimal policies.** ϵ -greedy keeps exploring forever if ϵ stays finite, which is wasteful in simple environments. A policy-gradient agent is stochastic but **can converge toward a deterministic policy**: actions that work get picked more often, creating momentum toward the optimum. You do not need to guess how fast to decay ϵ .
- **Direct optimisation of what you care about:** the performance $J(\theta)$, not a proxy.

The two weaknesses - and the cheap fixes

Problem	Why it happens	Fix from the video
High variance between episodes	Each visit to a state can choose a different action, leading to radically different returns	Scale returns by a baseline (the episode's mean) and divide by the standard deviation ; this keeps step sizes sane
Sample inefficiency	Memory is wiped each episode: the agent starts from scratch apart from the new weights (Monte-Carlo style)	Let the agent play a batch of episodes before each update, so it visits states more than once

$$\hat{G}_t = \frac{G_t - \text{mean}(G)}{\text{std}(G)}$$

(Guard against a standard deviation of 0 - the code divides only if std > 0.)

FROM THE VIDEO The author reports that **increasing the number of episodes per update** is what took Space Invaders from “no learning at all” to measurable improvement. The batch size is an extra hyper-parameter: bigger batches $\Rightarrow$ faster, more reliable convergence per update, but fewer updates per game.

Value-based vs. policy-based at a glance

	Value-based (Q-learning, DQN)	Policy-based (REINFORCE)
Learns	$Q(s,a)$ – how good each action is	$\pi(a s)$ – the action probabilities directly
Exploration	ϵ -greedy (an extra hyper-parameter and schedule)	Built in: sampling from the softmax
Update timing	Every step (TD) from a replay batch	End of an episode (or batch of episodes) – Monte Carlo
Reuse of experience	High: replay memory	Low: samples used once, then discarded
Output at the end	Greedy policy from argmax Q	Stochastic policy (can sharpen to deterministic)
Main weakness	Needs an accurate value function; overestimation bias	High variance; sample-inefficient; finicky learning rates

BEYOND THE VIDEO The advanced methods the video points to next – **actor-critic** and **DDPG** – combine both columns: a policy network (the actor) trained using a learned value function (the critic) as a lower-variance weight in place of the raw return G_t . This also unlocks **continuous** action spaces, where Q-learning cannot work.

22

Lunar Lander with policy gradients (TensorFlow)

Video: 1:55 – 2:21 · Part IV · Policy gradients

Goal: land a spaceship on the moon. The observation is not an image but an **8-number vector**; there are **4 actions**. You need the Box2D extension (pip install box2d-py). The whole agent lives in one class.

Network: input → 64 → 64 → logits → softmax (reconstructed from the narration)

```
class PolicyGradientAgent(object):
    def __init__(self, lr, gamma=0.99, n_actions=4, l1_size=64, l2_size=64, input_dims=8, ckpt_dir='tmp/checkpoints'):
        self.lr, self.gamma = lr, gamma
        self.n_actions, self.input_dims, self.l1_size, self.l2_size = n_actions, input_dims, l1_size, l2_size
        self.action_space = [i for i in range(n_actions)]
        self.state_memory, self.action_memory, self.reward_memory = [], [], []
        self.sess = tf.Session()
        self.build_net()
        self.sess.run(tf.global_variables_initializer())
        self.saver = tf.train.Saver()
        self.checkpoint_file = os.path.join(ckpt_dir, 'policy_network.ckpt')

    def build_net(self):
        xav = tf.contrib.layers.xavier_initializer() # keeps layer scales comparable
        with tf.variable_scope('parameters'):
            self.input = tf.placeholder(tf.float32, shape=[None, self.input_dims], name='input') # 8-number state
            self.label = tf.placeholder(tf.int32, shape=[None, 1], name='label') # action taken
            self.G = tf.placeholder(tf.float32, shape=[None, 1], name='G') # discounted return
            l1 = tf.layers.dense(self.input, self.l1_size, activation=tf.nn.relu, kernel_initializer=xav)
            l2 = tf.layers.dense(l1, self.l2_size, activation=tf.nn.relu, kernel_initializer=xav)
            l3 = tf.layers.dense(l2, self.n_actions, activation=None, kernel_initializer=xav) # logits
            self.actions = tf.nn.softmax(l3, name='probabilities') # pi(a|s)
        with tf.variable_scope('loss'):
            neg_log_prob = tf.nn.sparse_softmax_cross_entropy_with_logits(logits=l3, labels=self.label)
            loss = neg_log_prob * self.G
        self.train_op = tf.train.AdamOptimizer(self.lr).minimize(loss)
```

Acting, remembering, learning, main loop

```
def choose_action(self, observation):
    probs = self.sess.run(self.actions, feed_dict={self.input: observation[np.newaxis, :]}[0])
    return np.random.choice(self.action_space, p=probs) # sample from the distribution

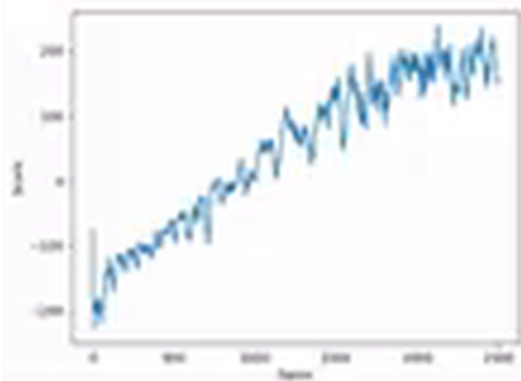
def store_transition(self, observation, action, reward):
    self.state_memory.append(observation); self.action_memory.append(action); self.reward_memory.append(reward)

def learn(self):
    state_memory, action_memory = np.array(self.state_memory), np.array(self.action_memory)
    reward_memory = np.array(self.reward_memory)
    G = np.zeros_like(reward_memory)
    for t in range(len(reward_memory)): # discounted sum of rewards AFTER step t
        G_sum, discount = 0, 1
        for k in range(t, len(reward_memory)):
            G_sum += reward_memory[k] * discount
            discount *= self.gamma
        G[t] = G_sum
    std = np.std(G) if np.std(G) > 0 else 1 # guard against std = 0
    G = (G - np.mean(G)) / std
    self.sess.run(self.train_op, feed_dict={self.input: state_memory, self.label: action_memory, self.G: G})
    self.state_memory, self.action_memory, self.reward_memory = [], [], [] # forget the episode

# main
agent = PolicyGradientAgent(lr=0.0005, gamma=0.99)
env = gym.make('LunarLander-v2'); score_history = []
for i in range(2500):
    done, score, observation = False, 0, env.reset()
    while not done:
        action = agent.choose_action(observation)
        observation_, reward, done, info = env.step(action)
        agent.store_transition(observation, action, reward)
        observation = observation_; score += reward
    score_history.append(score); agent.learn(); agent.save_checkpoint()
```

Notes & results

- **Hyper-parameters:** $\alpha = 0.0005$, $\gamma = 0.99$, two hidden layers of 64 units, Xavier initialisation, Adam.
- **Result:** after $\approx 2,500$ episodes the average score rises from ≈ -200 to ≈ 200 and above – the gym’s definition of “solved”.
- **Learn after every episode**, then clear the memory so rewards cannot leak into the next episode.
- **Debug diary** (the instructor’s typos, a useful list): missing input_dims attribute; name= not label= for placeholders; the environment id has no dash; method name store_transition.



Frame from the video – score vs. episode over 2,500 episodes.

BEYOND THE VIDEO Recent gym/gymnasium releases have renamed the environment (LunarLander-v3) and tf.contrib no longer exists in TensorFlow 2; use tf.keras.initializers.GlorotUniform() (Xavier) and Keras layers if you port this.

23

Policy gradients on Space Invaders

Video: 2:21 – 2:34 · Part IV · Policy gradients

Same algorithm, harder input: instead of an 8-number vector the observation is a **pixel image**, so the policy network starts with a CNN. The tutorial also introduces the batching idea from Section 21.

Network

Layer	Settings	Notes
Conv 1	32 filters, 8×8, stride 4	Xavier initialiser → batch-norm ($\epsilon = 1e-5$) → ReLU
Conv 2	64 filters, 4×4, stride 2	same pattern
Conv 3	128 filters, 2×2, stride 1	same pattern
Flatten → Dense	fc1 units (hyper-parameter), ReLU	convolution outputs are matrices; the dense layer needs a vector
Dense (logits)	6 units, no activation	softmax gives the probabilities; $-\ln \pi$ via sparse softmax cross-entropy $\times G$
Optimiser	RMSprop with tuned parameters	“Finicky”: expect to try several settings

Pre-processing and frame stacking (the same helper idea as in the DQN tutorials)

```
def preprocess(obs):
    return np.mean(obs[15:200, 30:125], axis=2).astype(np.uint8)    # crop + gray-scale

def stack_frames(stacked, frame, buffer_size=4):
    if stacked is None:
        stacked = np.zeros((buffer_size, *frame.shape))             # top of an episode
        for idx in range(buffer_size): stacked[idx, :] = frame        # fill with the first frame
    else:
        stacked[0:buffer_size - 1, :] = stacked[1:, :]              # drop the oldest ...
        stacked[buffer_size - 1, :] = frame                          # ... append the newest
    return stacked
```

What changes compared with Lunar Lander

- **Four stacked frames.** A single image has no sense of motion; and gym's Atari library repeats frames for 1–3 steps, so enough frames must be captured to see real movement. Four is the “magic number”.
- **Learn from a batch of episodes.** Policy gradients are Monte-Carlo: the agent normally discards all prior experience after one update. Queue up several episodes (the tutorial learns and saves every **10 games**) and compute returns **per episode** so rewards from one game never spill into the next.
- **Scale the returns** (mean/std) to reduce variance.
- **Logging:** print the running average of the last 20 games every 20 episodes; every other episode print episode number and score.

Result

The learning curve (average reward) shows oscillations up and down – “perfectly normal” – with an overall upward trend of roughly **+100 points**. That will not win any awards, but it is clear evidence of learning. The author stresses that this family of algorithms is **notoriously finicky about learning rates**; he did not tune heavily and hands the tuning to the viewer.

KEY IDEA Checklist when policy gradients don't learn: (1) normalise returns; (2) learn from more episodes per update; (3) lower/raise the learning rate by 3–10×; (4) make sure returns are computed within an episode; (5) check the mean-reward *trend* over many episodes, not single games.

24

Comparison, decision guide & hyper-parameter cheat-sheet

Video: synthesis of the whole course · Part V · Wrap-up

Method	Learns	Policy	Updates	Representation	Used on	Main caution
Q-learning	$Q(s,a)$	off	each step	table	Grid World	small discrete state space
SARSA	$Q(s,a)$	on	each step	table	CartPole (bucketed)	depends on exploration; discretisation
Double Q	Q_1, Q_2	off	each step	2 tables	CartPole (bucketed)	slower convergence
DQN	$Q(s,a;\theta)$	off	each step, replay batch	CNN / dense net	Breakout, Space Invaders	RAM, GPU, 2 networks
REINFORCE	$\pi(a s;\theta)$	on	episode(s)	NN + softmax	Lunar Lander, Space Invaders	variance, sample-inefficiency

Which method should I reach for?

- **Few discrete states, cheap simulation** (grid worlds, toy puzzles) → tabular **Q-learning**; try SARSA if safe behaviour while exploring matters; Double Q if rewards are noisy.
- **Continuous state but few dimensions** → discretise (as in CartPole) or switch to a small neural network.
- **Images / very large state spaces, discrete actions** → **DQN** with a CNN, replay memory and frame stacking.
- **Policy easier than value function, or you want a stochastic policy** → **policy gradients** (with return normalisation and batching).
- **Continuous actions** (e.g. bipedal walker, car racing) → beyond this course: DDPG, actor-critic.

Hyper-parameters as used in the course

Parameter	Role	Values seen
α (learning rate)	step size of updates	tabular 0.1 · DQN-TF 0.00025 · PG Lunar Lander 0.0005 · DQN-PyTorch 0.03 (as heard)
γ (discount)	value of future rewards	1.0 Grid World / Double Q · 0.9 SARSA · 0.99 Lunar Lander · ≈ 0.95 DQN-PyTorch
ϵ start → end	exploration	1.0 → 0 (Grid World, halfway) · 1.0 → 0.01 (Breakout) · 1.0 → 0.05 (Space Invaders)
Memory size	replay capacity	25,000 (Breakout, ≈ 48 GB RAM) · 5,000 (Space Invaders)
Batch size	transitions per update	32 (larger = slower, little gain)
Frames stacked	motion information	4 (Breakout, PG Space Invaders) · 3 (DQN Space Invaders)
Target replacement	refresh target net	optional; author left it off in Space Invaders (it hurt)
Games / episodes	training length	50 (SI-DQN) · 200 (Breakout) · 2,500 (Lunar Lander) · 50,000 (Grid World, SARSA) · 100,000 (Double Q)
Episodes per PG update	variance control	1 (Lunar Lander) · 10 (Space Invaders)

Troubleshooting

Symptom	Likely causes to check
Score never improves	ϵ decays too fast/slow; reward not what you think; terminal flag not handled; learning rate too high
Score improves then collapses	target-network replacement; too-small replay memory; learning-rate too high; random-seed luck (run again)
Out of memory	replay memory too large for RAM (store frames as uint8); batch too big for GPU
Extremely slow training	not using the GPU; rendering every frame; too large a batch
Weird behaviour at start	all Q equal → argmax picks the first action; use optimistic values or random pre-fill

25

Practice: exercises, quiz & where next

Video: 3:54 – 3:55 · Part V · Wrap-up

Ten exercises (easy → hard)

1. **Grid World:** add a third teleporter or change its destination. Predict the new optimal return, then verify by training.
2. **γ matters:** rerun Grid World Q-learning with $\gamma = 0.9$. Does it still find the shortcut? What changes in the numbers?
3. **Pessimistic start:** initialise Q to -50 instead of 0 and use a tiny ϵ . What happens to exploration, and why?
4. **SARSA on Grid World:** implement it and compare its learning curve with Q-learning (same seeds).
5. **Maximization bias:** build an environment with one noisy-reward state and show Double Q-learning beats Q-learning there.
6. **CartPole buckets:** try 6, 10 and 20 buckets per variable. Which learns fastest, which ends highest?
7. **Target network ablation:** in Space Invaders compare replace = None, 1,000 and 10,000. Do you reproduce the author's nose-dive?
8. **Memory budget:** store frames as uint8 and measure the RAM saving for a 25,000-transition memory.
9. **PG variance:** Lunar Lander with and without return normalisation. Plot both.
10. **PG batching:** update every 1, 5 and 10 episodes on Space Invaders; which is steadiest?

Self-quiz (answers in italics)

#	Question	Answer
1	What does $Q(s,a)$ estimate?	<i>The expected discounted return of taking action a in state s and following the policy afterwards.</i>
2	Q-learning vs SARSA in one sentence?	<i>Q-learning's target uses the best next action (off-policy); SARSA's uses the next action actually chosen (on-policy).</i>
3	Why stack frames?	<i>One frame carries no motion information; several consecutive frames show direction and speed.</i>
4	Why two networks in DQN?	<i>One selects/evaluates the current state, the other (target) gives the maximal-action value; this reduces bias in the estimates.</i>
5	Why sample replay memory randomly?	<i>To break correlations between consecutive transitions and avoid oscillation / being stuck in one corner of parameter space.</i>
6	What does optimistic initialisation do?	<i>Makes every untried action look better than the disappointing real reward, so the agent explores without randomness.</i>
7	Why can't a Q-table solve Breakout?	<i>The state space (pixel images) is far too large; you need function approximation.</i>
8	Why is ϵ kept above 0 in complex games?	<i>You can never be sure the strategy is optimal; occasional exploration tests the model.</i>
9	Why normalise returns in policy gradients?	<i>To reduce variance so update step sizes don't swing wildly between episodes.</i>
10	Why is policy-gradient ascent, not descent?	<i>It maximises performance $J(\theta)$; (the code minimises $-\ln \pi \cdot G$, which is the same thing).</i>
11	What is the Markov property?	<i>The next state depends only on the current state and action, not on the earlier history.</i>
12	What is maximization bias?	<i>Taking a max over noisy estimates is positively biased; Double Q-learning separates selection from evaluation to fix it.</i>

Where to go next (the video's closing advice)

You now have a solid foundation: Q-learning, policy gradients, SARSA, Double Q-learning and how to build your own environment. The instructor points to the frontier topics: **deep deterministic policy gradients (DDPG)**, **actor-critic** methods and **behavioural cloning**. They matter most for **continuous action spaces** – everything in this course used discrete actions, but environments like bipedal walker and car racing need continuous actions, which Q-learning cannot handle. Recommended reading mentioned in the video: Sutton & Barto, *Reinforcement Learning: An Introduction*; the author's GitHub (code for every tutorial) and neuralnet.ai blog posts.

R

Reference: glossary, formulas & modern-library notes

Video: quick lookup · Part V · Wrap-up

Glossary

Term	Meaning	Term	Meaning
Action value $Q(s,a)$	expected return of a in s, then following the policy	Model-free	no transition model needed
Agent	the learner/decision maker	Off-policy	learn about a policy different from the behaviour one
Bellman equation	recursive definition of value	On-policy	learn about the policy being followed
Bootstrapping	estimates built from estimates	Policy π	distribution over actions given state
Discount γ	weight of future rewards	Return G_t	discounted sum of future rewards
Episode	one start-to-terminal run	Softmax	turns scores into probabilities summing to 1
ϵ-greedy	random with prob. ϵ , else greedy	State space S^+	all states incl. terminal
Experience replay	random batches from stored transitions	Target network	copy of Q-net used for the max term
Logits	raw network outputs before softmax	TD error	target – current estimate
Markov property	future depends only on present state/action	Terminal state	ends episode; value 0

Formula sheet

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$
$v_{\pi}(s) = \sum_a \pi(a s) \sum_{s',r} p(s',r s,a) [r + \gamma v_{\pi}(s')]$
$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$
$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$
$Q_1(s,a) \leftarrow Q_1(s,a) + \alpha [r + \gamma Q_2(s', \arg\max_{a'} Q_1(s',a')) - Q_1(s,a)]$
$y = r + \gamma \max_{a'} Q_{\text{target}}(s',a') \quad (y = r \text{ if } s' \text{ is terminal})$
$\theta_{t+1} = \theta_t + \alpha G_t \frac{\nabla_{\theta} \pi(A_t S_t, \theta_t)}{\pi(A_t S_t, \theta_t)} = \theta_t + \alpha G_t \nabla_{\theta} \ln \pi(A_t S_t, \theta_t)$

Modern-library notes (beyond the video)

- **Gym → Gymnasium.** The maintained fork; reset() returns (obs, info), step() returns (obs, reward, terminated, truncated, info). Atari is provided by ale-py (ALE/...-v5 ids); Box2D via pip install gymnasium[box2d].
- **TensorFlow 1 → 2.** Sessions, placeholders and tf.contrib are gone from core TF 2; port the TF code with Keras (tf.keras.layers, GradientTape) or use PyTorch.
- **Code in this guide** is reconstructed from narration; the instructor's own repository (linked under the video) is the authority. Tabular listings were executed here and reproduce the reported –11 optimum.
- **Auto-captions** may mis-hear numbers; values flagged “as heard” should be double-checked.